

1. Programmation Dynamique

1.1. Résumé Exécutif

La programmation dynamique est une méthode de résolution de problèmes complexes en les décomposant en sous-problèmes plus simples. Son principe fondamental repose sur la résolution de chaque sous-problème une seule fois et le stockage de ses résultats, généralement dans une table ou une table de hachage, afin d'éviter des calculs redondants. Cette technique est particulièrement efficace pour les problèmes présentant des sous-problèmes qui se chevauchent.

Le document analyse cette approche à travers deux exemples principaux : le problème du sac à dos (Knapsack) et l'optimisation d'un partitionnement de plages (Range Partitioning). Pour le problème du sac à dos, il compare l'inefficacité d'une approche par force brute, de complexité exponentielle, à la solution pseudo-polynomiale offerte par la programmation dynamique via les équations de récurrence de Bellman.

Une généralisation clé est présentée : tout programme dynamique peut être modélisé comme un problème de recherche du plus court ou du plus long chemin dans un Graphe Orienté Acyclique (DAG). Cette abstraction permet de concevoir des solveurs génériques. Un tel framework en Java est d'ailleurs détaillé, structuré autour des concepts d'État (*State*), de Transition (*Transition*) et de Modèle (*Model*), démontrant comment implémenter une solution réutilisable pour différents problèmes de programmation dynamique.

1.2. Le Problème du Sac à Dos (Knapsack Problem)

Définition du Problème

Le problème du sac à dos consiste à maximiser la valeur totale d'un ensemble d'objets que l'on peut placer dans un sac, tout en respectant une contrainte de capacité de poids maximale.

- **Objectif :** Maximiser $\sum v_i x_i$
- **Contrainte :** $\sum w_i x_i \leq C$
 - v_i est la valeur de l'objet i .
 - w_i est le poids de l'objet i .
 - x_i est une variable binaire (0 ou 1) indiquant si l'objet i est sélectionné.
 - C est la capacité maximale du sac.

Approche par Force Brute

Une approche naïve consiste à énumérer toutes les combinaisons possibles d'objets.

- **Méthode :** Essayer toutes les solutions possibles. Pour n objets, il existe 2^n solutions.
- **Complexité :** La complexité temporelle est exponentielle, rendant cette approche impraticable pour des ensembles d'objets de taille moyenne. Par exemple, avec $n = 50$ et un temps de test de 1 ms par solution, le calcul prendrait plus de 30 000 ans.

Approche par Programmation Dynamique

L'idée centrale de la programmation dynamique est d'éviter de recalculer la solution pour des sous-problèmes déjà résolus (états équivalents).

- **Équations de Récurrence de Bellman** : On définit $O(k, j)$ comme la valeur optimale pour une capacité k en utilisant les objets de $\{1, \dots, j\}$.
 - **Cas Général** : Si le poids de l'objet j (w_j) est inférieur ou égal à la capacité restante k : $O(k, j) = \max(O(k, j-1), v_j + O(k-w_j, j-1))$ (On compare la solution sans prendre l'objet j avec la solution en le prenant).
 - **Cas où $w_j > k$** : L'objet j ne peut pas être pris. $O(k, j) = O(k, j-1)$
 - **Cas de Base** : Avec aucun objet, la valeur est nulle. $O(k, 0) = 0$ pour toute capacité k .

Implémentation et Complexité

Deux méthodes principales d'implémentation sont présentées pour stocker les résultats des sous-problèmes.

- **Implémentation avec Table** :
 - **Structure** : Un tableau à deux dimensions (index de l'objet, capacité restante).
 - **Complexité** : Le temps et l'espace sont en $\Theta(C \cdot n)$.
- **Implémentation avec Cache (Memoization)** :
 - **Structure** : Une table de hachage (par exemple `HashMap<Pair<Integer, Integer>, Integer>`) qui stocke la valeur de l'objectif pour chaque état (index, capacité résiduelle).
 - **Complexité** : Le temps et l'espace sont en $O(C \cdot n)$.

Analyse de la Complexité : La complexité en $\Theta(C \cdot n)$ est qualifiée de **pseudo-polynomiale**. Elle est polynomiale par rapport à la valeur numérique de la capacité C , mais exponentielle par rapport à la taille de l'entrée (le nombre de bits nécessaires pour représenter C , soit $\log(C)$). Le problème du sac à dos est donc **faiblement NP-difficile (Weakly NP-Hard)**, contrairement à des problèmes comme le TSP qui sont NP-difficiles au sens fort.

Index	0	1	2	3	4
Value	1	6	18	22	28
Weight	2	3	5	6	7

Tableau d'exemple d'objets pour le problème du sac à dos.

1.3.Optimisation d'un Partitionnement de Plages (Range Partitioning)

Définition du Problème

Étant donné une séquence de nombres non négatifs $S = [s_1, \dots, s_n]$ et un entier k , l'objectif est de partitionner S en k plages contiguës (ou moins) de manière à minimiser la somme maximale de n'importe quelle plage.

- **Exemple :** Pour $S = [1, 2, 3, 4, 5, 6, 7, 8, 9]$ et $k = 3$, une partition optimale est $[1, 2, 3, 4, 5], [6, 7], [8, 9]$. La somme maximale est 17.

Formulation par Programmation Dynamique

- **État :** $O(i, l)$ représente la valeur optimale (la somme maximale minimisée) pour la sous-séquence $[s_1, \dots, s_i]$ en utilisant au plus l partitions.
- **Équation de Récurrence :** La valeur $O(i, l)$ est obtenue en trouvant le meilleur point de césure j pour la dernière partition $[s_{j+1}, \dots, s_i]$. $O(i, l) = \min_{j \in [1..i-1]} \max(s_{j+1} + \dots + s_i, O(j, l-1))$
- **Cas de Base :**
 - Avec une seule partition ($l=1$) : $O(i, 1) = s_1 + \dots + s_i$
 - Pour une séquence d'un seul élément ($i=1$) : $O(1, l) = s_1$ pour tout $l \geq 1$.
- **Solution Finale :** La solution optimale pour le problème complet est $O(n, k)$.

1.4.Généralisation de la Programmation Dynamique via les Graphes

Le Principe Fondamental

Un concept unificateur puissant est que **chaque programme dynamique peut être réduit à un problème de plus court (pour la minimisation) ou de plus long (pour la maximisation) chemin dans un Graphe Orienté Acyclique (DAG).**

- Le problème du plus long chemin est NP-difficile dans un graphe général, mais il peut être résolu en temps linéaire dans un DAG en utilisant la programmation dynamique.
- Le DAG représente un encodage compact de l'espace des solutions, où le chemin optimal correspond à la solution optimale.

Équation de Récurrence pour le Plus Long Chemin

Soit $L(i)$ le plus long chemin depuis le nœud i jusqu'à un nœud terminal t . L'équation de récurrence est :

$$L(i) = \max_{j \in \text{succ}(i)} (\text{coût}(i, j) + L(j))$$

où $\text{succ}(i)$ est l'ensemble des successeurs du nœud i et $\text{coût}(i, j)$ est le poids de l'arête de i à j . Le calcul se fait en partant des nœuds terminaux et en remontant vers la source (bottom-up).

1.5.Un Framework Générique pour la Programmation Dynamique en Java

Une approche générique peut être construite pour résoudre une large classe de problèmes de programmation dynamique.

Architecture et Composants Clés

Le framework proposé est composé de trois classes principales :

1. **State** : Une classe abstraite représentant un nœud du DAG (un sous-problème). Elle doit être "hashable", c'est-à-dire implémenter des méthodes `hash()` et `isEqual()` pour pouvoir être utilisée comme clé dans une table de hachage.
2. **Transition** : Représente une arête du DAG. Elle contient un état successeur, la décision qui a mené à cette transition et la valeur (ou coût) associée.
3. **Model** : Une classe abstraite qui définit la structure du problème spécifique :
 - o `getRootState()` : L'état initial.
 - o `isBaseCase(S state)` : Vérifie si un état est un cas de base.
 - o `getBaseCaseValue(S state)` : Retourne la valeur pour un cas de base.
 - o `getTransitions(S state)` : Génère toutes les transitions possibles à partir d'un état donné.

Le Solveur `DynamicProgramming`

La classe `DynamicProgramming` est le solveur principal. Elle utilise une `HashMap<State, Double>` pour stocker la meilleure valeur objective trouvée pour chaque état. Elle prend un `Model` en paramètre et l'utilise pour explorer récursivement l'espace des états.

Reconstruction de la Solution

Une fois la table des valeurs optimales remplie (le calcul de l'objectif pour l'état racine étant terminé), la séquence de décisions qui mène à la solution optimale peut être reconstruite.

- **Méthode** : En partant de l'état racine, on examine ses successeurs. On choisit la transition dont la valeur, ajoutée à la valeur optimale de son état successeur (déjà calculée et stockée dans la table), correspond à la valeur optimale de l'état courant. Ce processus est répété de manière gloutonne jusqu'à atteindre un état de base.

1.6. Application et Exercices

Le document se conclut par une suggestion de projet : résoudre le **problème du voyageur de commerce** (**Traveling Salesperson Problem - TSP**) en utilisant le framework de programmation dynamique.

- **Indice sur l'État** : Un état pour le TSP peut être défini par un tuple contenant :
 1. La position actuelle du vendeur.
 2. L'ensemble des villes déjà visitées.

2.Évaluation Comparative des Algorithmes d'Optimisation

2.1.Résumé Exécutif

Ce document de synthèse analyse les méthodologies d'évaluation comparative (benchmarking) pour les algorithmes d'optimisation. Le défi principal consiste à comparer de manière rigoureuse plusieurs solveurs sur un ensemble d'instances de test afin de tirer des conclusions générales et quantifiables sur leur performance.

Les approches simplistes, telles que le calcul du temps d'exécution moyen ou le simple décompte des instances résolues, s'avèrent inadéquates. Le temps moyen est excessivement influencé par les instances les plus difficiles, tandis que le décompte des succès ignore les nuances de performance relative.

Pour une analyse plus fine, deux principales techniques graphiques sont recommandées :

1. **Le Graphique Cactus (Cactus Plot)** : Il visualise le nombre cumulé d'instances résolues en fonction du temps, offrant une vue détaillée de la vitesse de résolution. Son principal défaut est de perdre l'information sur la performance relative par instance.
2. **Le Profil de Performance (Performance Profile)** : Proposée par Dolan et Moré (2002), cette méthode est la plus robuste. Elle normalise le temps d'exécution de chaque solveur par rapport à un "meilleur solveur virtuel" pour chaque instance. Le graphique résultant montre la proportion d'instances qu'un solveur peut résoudre dans un facteur de temps τ par rapport au meilleur, offrant une vision combinée de l'efficacité et de la robustesse.

Au-delà du simple temps de résolution, l'analyse peut intégrer des métriques comme l'**écart d'optimalité (optimality gap)** pour évaluer les solutions obtenues avant l'atteinte de l'optimalité, ou l'**intégrale primale** pour mesurer la performance "anytime" d'un algorithme. Des visualisations comme les **SinaPlots** et les **diagrammes de dispersion** permettent des comparaisons plus détaillées.

Enfin, un avertissement critique est émis concernant l'agrégation de résultats normalisés : l'utilisation de la **moyenne arithmétique est une erreur fondamentale** qui mène à des conclusions contradictoires selon la base de normalisation choisie. La **moyenne géométrique** est la seule méthode correcte pour agréger de tels résultats, garantissant la cohérence des conclusions.

2.2.Le Défi de l'Évaluation Comparative (Benchmarking)

L'évaluation comparative vise à comparer un ensemble d'approches ou d'algorithmes (nommés solveurs, $\mathcal{S}$) conçus pour résoudre un problème d'optimisation. Cette comparaison s'effectue sur une collection d'instances de test ($\mathcal{P}$), supposées représentatives des problèmes futurs. Pour chaque solveur $s \in \mathcal{S}$ et chaque problème $p \in \mathcal{P}$, une mesure de performance, typiquement le temps de calcul $t_{p,s}$, est enregistrée.

Les objectifs principaux d'une telle évaluation sont :

- **Tirer des conclusions générales** sur l'efficacité des solveurs, à condition que les instances de test soient suffisamment représentatives.
- **Quantifier la vitesse** relative d'un solveur par rapport aux autres.
- **Estimer la probabilité** qu'un solveur soit le plus performant sur une nouvelle instance inconnue (ce qui nécessite des tests statistiques non abordés ici).

2.3.Méthodes d'Évaluation Inadéquates

Plusieurs approches intuitives pour comparer les solveurs se révèlent être trompeuses et doivent être évitées.

Proposition 1 : Temps de Calcul Moyen

Calculer le temps moyen d'exécution pour chaque solveur sur l'ensemble des instances est une méthode simple mais erronée pour deux raisons majeures :

- **Domination par les instances difficiles** : Une seule instance très difficile peut fausser complètement la moyenne, masquant une bonne performance sur toutes les autres instances.
- **Biais en cas d'instances non résolues** : Si l'on écarte les instances qu'un solveur ne parvient pas à résoudre dans le temps imparti, on introduit un biais en faveur des solveurs les plus robustes, au détriment de ceux qui sont potentiellement plus rapides sur les instances qu'ils résolvent.

Proposition 2 : Nombre d'Instances Résolues

Classer les solveurs simplement en fonction du nombre d'instances qu'ils ont résolues est également une mauvaise approche :

- **Granularité très faible** : Cette méthode ne fait pas de distinction entre un solveur qui résout une instance en une seconde et un autre qui la résout en une heure.
- **Perte de l'ordre relatif** : L'information cruciale sur quel solveur est le plus rapide pour une instance donnée est complètement perdue.

2.4.Méthodologies d'Analyse Recommandées

Des outils plus sophistiqués sont nécessaires pour une comparaison juste et informative.

Graphiques Cactus (Cactus Plots)

Le graphique Cactus offre une visualisation plus détaillée de la performance. Il représente, pour chaque solveur, le nombre cumulé (ou le pourcentage) d'instances résolues en fonction du temps d'exécution.

- **Formule** : La fonction du graphique Cactus pour un solveur s est définie par : $\kappa_s(\tau) = (1 / |\mathcal{P}|) * |\{p \in \mathcal{P} : t_{p,s} \leq \tau\}|$

- **Avantages** : Offre une vue fine de la distribution des temps de résolution. Un solveur dont la courbe monte rapidement vers la gauche est globalement plus rapide.
- **Inconvénients** : Comme le décompte des instances, cette méthode perd l'information sur la performance relative. On ne peut pas savoir si un solveur est légèrement ou massivement plus lent qu'un autre sur une instance spécifique.

Le graphique ci-dessous illustre un Cactus Plot pour trois solveurs A, B et C. On observe que le solveur C résout rapidement un grand nombre de petites instances, tandis que A et B sont plus lents au démarrage mais finissent par résoudre le même nombre total d'instances.

Profils de Performance (Performance Profiles)

Cette méthode, introduite par Dolan et Moré en 2002, est devenue un standard pour l'évaluation comparative des logiciels d'optimisation. Elle repose sur la normalisation des temps de calcul par rapport à un "meilleur solveur virtuel".

1. **Ratio de Performance** : Pour chaque problème p et chaque solveur s , on calcule le ratio de performance $r_{p,s} : r_{p,s} = t_{p,s} / \min\{t_{p,s'} : s' \in \mathcal{S}\}$. Ce ratio indique de combien de fois le solveur s est plus lent que le meilleur solveur pour l'instance p . Si s est le meilleur, $r_{p,s} = 1$.
2. **Profil de Performance** : Le profil de performance $\rho_s(\tau)$ d'un solveur s est la proportion de problèmes pour lesquels le ratio de performance $r_{p,s}$ est inférieur ou égal à un facteur τ . $\rho_s(\tau) = (1 / |\mathcal{P}|) * |\{p \in \mathcal{P} : r_{p,s} \leq \tau\}|$

Interprétation des Profils de Performance :

- **Efficacité ($\tau = 1$)** : La valeur $\rho_s(1)$ correspond à la fraction des instances pour lesquelles le solveur s est le plus rapide. Le solveur avec la plus haute valeur à $\tau=1$ est le plus susceptible d'être le plus performant.
- **Robustesse** : La valeur limite du profil de performance (pour un τ grand) indique la proportion totale des problèmes que le solveur est capable de résoudre. Le solveur avec la valeur limite la plus élevée est le plus robuste.
- **Compromis** : La forme générale de la courbe permet de visualiser le compromis efficacité/robustesse. Un solveur dont la courbe est systématiquement au-dessus des autres est supérieur.

Dans un exemple réel comparant les solveurs LANCELOT, LOQO, MINOS et SNOPT, le profil de performance montre que LOQO a la plus grande probabilité d'être le plus rapide ($\rho(1)$ est le plus élevé). Cependant, en autorisant un facteur de ralentissement plus grand ($\tau \geq 4$), LANCELOT devient plus robuste, résolvant plus de problèmes. L'utilisation d'une **échelle logarithmique** pour l'axe τ peut révéler que SNOPT devient encore plus robuste pour des temps de calcul très longs.

2.5.Évaluation au-delà du Temps d'Optimalité

Lorsque les solveurs ne peuvent pas prouver l'optimalité dans le temps imparti (timeout), d'autres métriques deviennent pertinentes.

L'Écart d'Optimalité (Optimality Gap)

L'écart d'optimalité γ mesure la distance relative entre la meilleure solution trouvée $\bar{z}$ et la valeur optimale connue (ou une borne inférieure) z^* . $\gamma = |\bar{z} - z^*| / |\bar{z}|$
 Un **profil d'écart (Gap Profile)** peut être tracé, montrant la proportion d'instances pour lesquelles un solveur atteint un écart d'optimalité inférieur à un seuil donné à la fin du temps imparti.

Comportement "Anytime" et Intégrale Primale

Un algorithme a un bon comportement "anytime" s'il trouve rapidement des solutions de haute qualité, même s'il est interrompu avant la fin. L'**intégrale primale** ($P(T)$) est une métrique qui quantifie cette capacité en intégrant l'écart primal (une version normalisée de l'écart d'optimalité) sur la durée d'exécution. Une valeur plus faible de $P(T)$ indique une meilleure qualité des heuristiques primales du solveur.

2.6.Outils de Visualisation Complémentaires

Graphiques SinaPlot

Le SinaPlot est une technique de visualisation qui affiche chaque point de donnée individuel tout en montrant leur distribution, ce qui le rend plus informatif qu'un diagramme en boîte (box-plot). Il est utile pour comparer des distributions de métriques comme le nombre de nœuds explorés par seconde.

Diagrammes de Dispersion (Scatter Plots)

Pour une comparaison directe entre deux algorithmes, un diagramme de dispersion est très efficace. En plaçant les performances de l'algorithme A sur l'axe des X et celles de l'algorithme B sur l'axe des Y pour chaque instance, on peut visualiser immédiatement lequel est le meilleur et dans quelle mesure. Les points au-dessus de la diagonale $y=x$ indiquent une meilleure performance pour A, et inversement.

2.7.Avertissement Critique : L'Agrégation des Résultats Normalisés

Une erreur fréquente et grave dans l'analyse de benchmarks est l'utilisation de la moyenne arithmétique pour agréger des résultats normalisés.

Le Piège de la Moyenne Arithmétique

Normaliser les temps d'exécution par rapport à un solveur de référence, puis calculer la moyenne arithmétique de ces ratios est une pratique incorrecte. Les conclusions qui en découlent dépendent entièrement du choix du solveur de référence, comme le montre l'exemple ci-dessous.

Benchmark	Processeur R	Processeur M	Processeur Z
E	417 (1.00)	244 (0.59)	134 (0.32)
F	83 (1.00)	70 (0.84)	70 (0.85)

H	66 (1.00)	153 (2.32)	135 (2.05)
I	39,449 (1.00)	33,527 (0.85)	66,000 (1.67)
K	772 (1.00)	368 (0.48)	369 (0.45)
Moyenne Arithmétique	(1.00)	(1.01)	(1.07)

Avec R comme référence, R semble être le gagnant.

Benchmark	Processeur R	Processeur M	Processeur Z
E	417 (1.71)	244 (1.00)	134 (0.55)
F	83 (1.19)	70 (1.00)	70 (1.00)
H	66 (0.43)	153 (1.00)	135 (0.88)
I	39,449 (1.18)	33,527 (1.00)	66,000 (1.97)
K	772 (2.10)	368 (1.00)	369 (1.00)
Moyenne Arithmétique	(1.32)	(1.00)	(1.08)

Avec M comme référence, M devient le gagnant. Les mêmes données mènent à des conclusions opposées.

La Solution : La Moyenne Géométrique

Comme démontré par Fleming et Wallace (1986), la **moyenne géométrique est la seule manière correcte d'agréger des résultats normalisés**. Elle produit une conclusion cohérente quel que soit le point de référence choisi pour la normalisation. En appliquant la moyenne géométrique aux données précédentes, les processeurs M et Z sont systématiquement identifiés comme étant supérieurs à R, indépendamment de la colonne de normalisation.

3.Algorithme Branch and Bound

3.1.Résumé Exécutif

L'algorithme *Branch and Bound* (Séparer et Évaluer) est une méthode d'optimisation avancée conçue pour résoudre des problèmes complexes, notamment ceux classés comme NP-difficiles, en évitant l'exploration exhaustive de l'espace des solutions. Contrairement à une approche par force brute qui examine toutes les possibilités, le Branch and Bound réduit considérablement l'espace de recherche en "élaguant" les branches de l'arbre de recherche qui ne peuvent pas contenir de solution optimale.

Le principe repose sur deux mécanismes clés :

1. **Branching (Séparation)** : Le problème initial est décomposé de manière récursive en sous-problèmes plus petits, formant un arbre de recherche.
2. **Bounding (Évaluation)** : Pour chaque sous-problème (nœud de l'arbre), une borne (supérieure pour la maximisation, inférieure pour la minimisation) est calculée. Cette borne représente la meilleure valeur que toute solution dans cette branche pourrait atteindre de manière optimiste.

Le moteur de l'algorithme est l'élagage : si la borne d'un sous-problème est moins bonne que la meilleure solution déjà trouvée, toute la branche issue de ce sous-problème est ignorée. L'efficacité de l'algorithme dépend de la qualité de la borne (elle doit être à la fois "serrée" et rapide à calculer) et de la stratégie d'exploration de l'arbre (DFS, BFS, Best-First). Des techniques comme la **relaxation de contraintes** (par exemple, la relaxation linéaire pour le problème du sac à dos) sont essentielles pour obtenir des bornes efficaces. L'algorithme est appliqué avec succès à des problèmes classiques tels que le Problème du Sac à Dos et le Problème du Voyageur de Commerce (TSP).

3.2.Le Principe Fondamental du Branch and Bound

Dépassement de la Recherche par Force Brute

Une approche par force brute pour un problème d'optimisation consiste à générer et évaluer toutes les solutions candidates. Cette méthode est souvent représentée par un arbre de recherche où chaque chemin de la racine à une feuille correspond à une solution potentielle. Cependant, la taille de cet arbre croît de manière exponentielle, rendant cette approche impraticable pour la plupart des problèmes réels. La question centrale que le Branch and Bound cherche à résoudre est : "Pouvons-nous réduire cet espace de recherche en coupant certaines branches ?"

Le Concept Clé : Élagage de l'Arbre de Recherche

L'idée générale est illustrée par une analogie : la recherche de la plus grosse gemme possible.

- Supposons que vous ayez déjà trouvé une gemme de taille 50 (votre meilleure solution actuelle).
- Vous évaluez deux zones de fouille potentielles (deux sous-problèmes) :

- **Zone A** : Une évaluation rapide et optimiste indique que la gemme y est *au mieux* de taille 40.
- **Zone B** : L'évaluation optimiste indique une gemme potentielle de taille 110.

La décision est évidente : il est inutile de creuser dans la Zone A, car aucune gemme trouvée ne pourra surpasser celle que vous possédez déjà. Vous pouvez "élaguer" cette branche de recherche et concentrer vos efforts sur la Zone B, plus prometteuse.

Formalisation du Processus d'Élagage

Pour un problème de maximisation P visant à maximiser un objectif obj :

1. **Solution réalisable** : On dispose d'une solution réalisable avec une valeur d'objectif obj^* (la meilleure trouvée jusqu'à présent, agissant comme une borne inférieure globale).
2. **Décomposition** : Le problème P est décomposé en sous-problèmes, par exemple $P = P_1 \cup P_2$.
3. **Borne supérieure** : On utilise une procédure $UB(P_i)$ qui calcule une borne supérieure optimiste pour la valeur de la meilleure solution dans le sous-problème P_i .
4. **Condition d'élagage** : Si $UB(P_1) \leq obj^*$, alors l'exploration du sous-problème P_1 peut être abandonnée, car aucune solution dans cette branche ne pourra améliorer la meilleure solution actuelle.

3.3.Calcul des Bornes par Relaxation

Le Rôle de la Relaxation des Contraintes

La clé du "Bounding" est d'obtenir une borne de manière rapide. Pour un problème de maximisation sous contraintes, une méthode efficace pour calculer une borne supérieure consiste à **relaxer** (ignorer) certaines de ses contraintes. La solution du problème relaxé, qui est plus simple à résoudre, fournit une évaluation optimiste de la solution du problème original.

Application au Problème du Sac à Dos (Knapsack)

Considérons le problème du sac à dos 0/1 suivant :

- **Maximiser** : $28x_1 + 30x_2 + 20x_3$
- **Sous la contrainte** : $4x_1 + 6x_2 + 4x_3 \leq 9$
- **Avec** : $x_i \in \{0, 1\}$

La Relaxation Linéaire (Intégralité)

Une technique de relaxation puissante est la **relaxation de la contrainte d'intégralité**. Au lieu d'exiger que les variables x_i soient binaires (0 ou 1), on les autorise à prendre n'importe quelle valeur réelle entre 0 et 1. Le problème relaxé devient :

- **Maximiser** : $28x_1 + 30x_2 + 20x_3$
- **Sous la contrainte** : $4x_1 + 6x_2 + 4x_3 \leq 9$
- **Avec** : $x_i \in [0, 1]$

Ce type de relaxation, où toutes les fonctions sont linéaires, est appelé **relaxation de programmation linéaire**.

Résolution Efficace de la Relaxation Linéaire

Le problème du sac à dos avec relaxation linéaire peut être résolu très efficacement par une approche gloutonne :

1. **Trier les objets** : Classer les objets par ordre décroissant du ratio valeur/poids (v_i/w_i).
2. **Remplir le sac** : Ajouter les objets dans cet ordre jusqu'à ce que le sac soit plein.
3. **Objet critique** : Pour le premier objet qui ne rentre pas entièrement (l'objet "critique" j), prendre la fraction qui remplit la capacité restante.

La formule de la borne supérieure (UB) est : $UB = \sum (v_i \text{ pour } i < j) + (C - \sum (w_i \text{ pour } i < j - 1)) / w_j * v_j$

Exemple : Pour un sac de capacité $C=14$, avec les objets triés :

Index	v	w	v/w	$\sum w$	x^*
0	28	7	4	7	1
1	22	6	3.66	13	1
2	18	5	3.6	18	1/5
3	6	2	3	20	0
4	1	1	1	21	0

L'objet 2 est l'objet critique. La borne supérieure calculée est : $28 + 22 + 1/5 * 18 = 53.6$.

3.4.Stratégies de Recherche et d'Exploration ("Branching")

La manière dont l'arbre de recherche est exploré a un impact significatif sur les performances de l'algorithme. Les nœuds non explorés sont conservés dans une collection de "nœuds ouverts".

Comparaison des Stratégies

Stratégie	Principe	Avantages	Inconvénients
Depth-First Search (DFS)	Explore le nœud le plus profond et le plus à gauche en premier (LIFO).	Mémoire proportionnelle à la hauteur de l'arbre (généralement linéaire).	Moins efficace pour prouver l'optimalité ou trouver rapidement une bonne première solution.
Breadth-First Search (BFS)	Explore les nœuds niveau par niveau (FIFO).	Meilleur pour réduire l'écart d'optimalité.	Peut consommer une grande quantité de mémoire.
Best-First Search	Explore le nœud le plus "prometteur" (avec la meilleure borne).	Très efficace si la procédure de calcul de borne est bonne.	Pas de contrôle sur le nombre de nœuds ouverts, risque de consommation mémoire élevée.

Stratégies Hybrides

Une approche efficace consiste à combiner les stratégies :

1. Commencer avec un **DFS** pour trouver rapidement une bonne solution réalisable, ce qui établit une borne de référence solide pour l'élagage.
2. Continuer avec un **Best-First Search** pour explorer les zones les plus prometteuses et prouver l'optimalité.

Le code Java illustre une telle stratégie : un nœud est sélectionné via une file de priorité (Best-First), puis l'algorithme "plonge" de manière gloutonne vers une feuille (comme un DFS) pour trouver une solution candidate.

3.5.Implémentation en Pratique (Java)

Une implémentation générique de Branch and Bound s'articule autour des concepts suivants :

- **Structure Générique** : Une boucle principale qui retire un nœud de la collection des nœuds ouverts tant qu'elle n'est pas vide.
- **Logique d'élagage** : Un nœud n'est exploré que si sa borne est meilleure que la meilleure solution connue (`n.lowerBound() < upperBound` pour une minimisation).
- **Mise à jour** : Si un nœud est une solution candidate et améliore la meilleure solution connue, cette dernière est mise à jour.

Interfaces Clés : Node et OpenNodes

Pour garantir la modularité, le code s'appuie sur des interfaces :

- `interface Node<T>` : Représente un nœud dans l'arbre de recherche. Doit fournir des méthodes pour vérifier s'il est une solution, calculer sa borne, et générer ses enfants.
- `interface OpenNodes<N extends Node>` : Représente la collection des nœuds ouverts. Ses implémentations déterminent la stratégie de recherche (e.g., une `Stack` pour DFS, une `PriorityQueue` pour Best-First).

3.6.L'Écart d'Optimalité ("Optimality Gap")

L'écart d'optimalité est une mesure de la progression de l'algorithme vers la solution optimale.

Définition et Calcul

Pour un problème de maximisation :

- **Borne Inférieure (LB - Lower Bound)** : La valeur de la meilleure solution réalisable trouvée jusqu'à présent.
- **Borne Supérieure Globale (UB - Upper Bound)** : Le maximum des bornes supérieures de tous les nœuds ouverts (non explorés).
- **Écart d'Optimalité** : $UB - LB$

La recherche est terminée et l'optimalité est prouvée lorsque l'écart est nul (c'est-à-dire $UB = LB$).

Impact des Stratégies de Recherche sur l'Écart

- **DFS** est généralement plus efficace pour trouver rapidement une bonne **LB**.
- **BFS** ou **Best-First** sont généralement plus efficaces pour faire baisser la **UB** globale et donc "fermer" l'écart.

3.7. Application au Problème du Voyageur de Commerce (TSP)

Le TSP consiste à trouver le circuit le plus court qui visite un ensemble de villes une seule fois. C'est un problème de minimisation.

Modélisation du TSP

Le TSP peut être modélisé avec deux contraintes principales sur les arêtes x_e sélectionnées :

1. **Contrainte de Connectivité** : Les arêtes sélectionnées doivent former un cycle unique (un tour).
2. **Contrainte de Degré** : Chaque ville (nœud) doit avoir exactement deux arêtes incidentes ($\sum x_e = 2$).

Techniques de Relaxation pour le TSP

Pour obtenir une borne inférieure (LB) pour le TSP, on peut relaxer l'une de ces contraintes.

Arbre Couvrant de Poids Minimum (MST) + Arête la Moins Chère

- **Relaxation** : On ignore la contrainte de degré. Le problème relaxé est de trouver un sous-graphe couvrant avec n arêtes (un arbre + une arête).
- **Calcul** : Poids du MST + poids de l'arête la moins chère non présente dans le MST.

Arbre de Poids Minimum

- **Relaxation** : Une version plus forte. On choisit un nœud arbitraire (ex: 0). On calcule le MST sur tous les autres nœuds, puis on connecte le nœud 0 via ses deux arêtes les moins chères.
- **Propriétés** : Chaque nœud a un degré d'au moins 2, sauf potentiellement le nœud 0. C'est une borne inférieure plus "serrée" que la précédente. Son calcul a une complexité de $O(E \log V)$.

Couplage de Poids Minimum

- **Relaxation** : On ignore la contrainte de connectivité, en ne gardant que la contrainte de degré.
- **Calcul** : On cherche un ensemble d'arêtes où chaque nœud a un degré de 2, au coût minimal.
- **Inconvénient** : La solution peut être composée de plusieurs sous-tours déconnectés.

3.8.Synthèse et Conclusion

L'algorithme Branch and Bound est une technique puissante pour les problèmes d'optimisation combinatoire, mais son succès n'est pas automatique. Son efficacité repose sur plusieurs piliers :

- **Qualité des Bornes** : Il est crucial de développer des procédures de calcul de bornes qui soient à la fois "serrées" (proches de la valeur optimale) et rapides à calculer.
- **Algorithmes de Relaxation** : La relaxation de contraintes est la méthode la plus courante pour obtenir de bonnes bornes, s'appuyant souvent sur des algorithmes polynomiaux bien connus (ex: Kruskal pour le MST).
- **Implémentation et Stratégie** : Le choix de la structure de données pour les nœuds ouverts (qui dicte la stratégie de recherche) et une implémentation soignée sont déterminants.

Comme le dit l'adage, "**L'optimisation combinatoire est l'art de la relaxation**". C'est la capacité à simplifier intelligemment un problème pour guider la recherche qui fait toute la force de cette approche, initiée par les travaux de pionniers comme Ailsa Land, Alison Doig, Georges Dantzig et Richard E. Bellman.

4. Relaxation Lagrangienne

4.1. Résumé Exécutif

La relaxation lagrangienne est une technique générique et puissante en optimisation, conçue pour calculer des bornes inférieures de haute qualité pour des problèmes complexes, souvent NP-difficiles. Le principe fondamental consiste à transformer un problème difficile, caractérisé par plusieurs ensembles de contraintes, en un problème plus simple à résoudre. Ceci est accompli en "relaxant" une ou plusieurs contraintes complexes et en les intégrant dans la fonction objectif sous forme de pénalité. Cette pénalité est pondérée par un multiplicateur de Lagrange (noté λ ou π).

La solution du problème relaxé, qui est plus facile à calculer, fournit une borne inférieure à la solution optimale du problème initial. L'objectif ultime est de trouver les valeurs des multiplicateurs de Lagrange qui maximisent cette borne inférieure, la rendant ainsi la plus "serrée" possible. Ce processus de maximisation est connu sous le nom de problème dual lagrangien.

Le document explore cette méthode à travers deux applications principales :

1. **Le Problème du Plus Court Chemin avec Contrainte de Ressource (RCSPP)** : La contrainte de ressource (par exemple, le temps) est relaxée, transformant le problème en un simple calcul de plus court chemin avec des poids d'arêtes modifiés.
2. **Le Problème du Voyageur de Commerce (TSP)** : Les contraintes de degré des nœuds sont relaxées, permettant de calculer une borne inférieure basée sur un problème de 1-arbre minimum, qui est calculable en temps polynomial.

Pour trouver les multiplicateurs optimaux, des méthodes comme l'algorithme du sous-gradient sont couramment utilisées. La qualité de la borne lagrangienne est équivalente à celle de la relaxation linéaire du problème. De plus, cette technique peut fournir une preuve d'optimalité si la borne inférieure calculée coïncide avec la valeur d'une solution réalisable trouvée.

4.2. Principe Fondamental de la Relaxation Lagrangienne

L'intuition de la relaxation lagrangienne est de simplifier un problème d'optimisation difficile en se débarrassant temporairement des contraintes qui le rendent complexe.

Considérons un problème difficile de la forme :

- **Maximiser** : obj
- **Sujet à** : Contrainte 1 ET Contrainte 2

Si le problème devient facile à résoudre en l'absence de la Contrainte 1, nous pouvons le transformer. La Contrainte 1 est déplacée dans la fonction objectif en pénalisant sa violation. Le nouveau problème, plus simple, s'écrit :

- **Maximiser** : $\text{obj} + \lambda * \text{violation}(\text{Contrainte 1})$
- **Sujet à** : Contrainte 2

La solution de ce problème "relaxé" fournit une borne (ici, une borne inférieure pour un problème de minimisation) sur la valeur optimale du problème initial. La tâche principale devient alors de trouver la meilleure valeur pour le multiplicateur λ afin de maximiser cette borne inférieure. C'est le problème **dual lagrangien**.

4.3.Application au Problème du Plus Court Chemin avec Contrainte de Ressource (RCSPP)

Définition du Problème

Le RCSPP est un problème NP-difficile qui cherche à trouver un chemin d'un point de départ s à un point d'arrivée e dans un graphe. Chaque arc (i, j) possède un coût c_{ij} et une consommation de ressource t_{ij} (par exemple, un temps de parcours). L'objectif est de minimiser le coût total du chemin tout en respectant une contrainte de ressource globale T .

La formulation mathématique est :

- **Minimiser** : $\sum (c_{ij} \cdot x_{ij})$
- **Sujet à** :
 - Contraintes de conservation du flux (assurant la formation d'un chemin valide).
 - $\sum (t_{ij} \cdot x_{ij}) \leq T$ (la contrainte de ressource).
 - $x_{ij} \in \{0, 1\}$ (variable binaire indiquant si un arc est dans le chemin).

Formulation de la Relaxation

Sans la contrainte de ressource, le problème se réduit à un simple problème de plus court chemin, qui est facile à résoudre (par exemple, avec l'algorithme de Dijkstra). La relaxation lagrangienne consiste donc à relaxer cette contrainte de ressource.

La fonction lagrangienne $L(\lambda)$ est définie pour un multiplicateur $\lambda \geq 0$: $L(\lambda) = \min [\sum (c_{ij} \cdot x_{ij}) + \lambda (\sum (t_{ij} \cdot x_{ij}) - T)]$

En réarrangeant les termes, on obtient : $L(\lambda) = \min [\sum ((c_{ij} + \lambda t_{ij}) \cdot x_{ij})] - \lambda T$

Pour une valeur de λ donnée, le calcul de $L(\lambda)$ revient à trouver le plus court chemin dans le même graphe, mais avec des poids d'arête modifiés $(c_{ij} + \lambda t_{ij})$. Le résultat de ce calcul de plus court chemin, moins λT , est une borne inférieure sur le coût optimal du RCSPP.

Exemple de Calcul de Borne Inférieure : Avec $T = 10$ et en choisissant $\lambda = 2$, les poids $(c_{ij} + \lambda t_{ij})$ sont calculés pour chaque arc. Le plus court chemin avec ces nouveaux poids a un coût de 35. La borne inférieure est alors $L(\lambda=2) = 35 - 2 * 10 = 15$.

Calcul de la Meilleure Borne Inférieure (Dual Lagrangien)

Le but est de trouver le λ qui maximise $L(\lambda)$, c'est-à-dire $L^* = \max (\lambda \geq 0) L(\lambda)$.

La fonction $L(\lambda)$ est la borne inférieure de plusieurs fonctions linéaires (une pour chaque chemin possible dans le graphe), ce qui en fait une fonction **convexe et linéaire par morceaux**. Le but est de trouver le pic de cette fonction.

Plusieurs méthodes existent pour trouver le λ optimal :

- **Approche par force brute** : Énumérer tous les chemins réalisables P , calculer $c_P + \lambda(t_P - T)$ pour chacun et prendre le minimum. Cette approche est impraticable car le nombre de chemins est exponentiel.

Chemin (P)	Coût (cP)	Temps (tP)	$c_P + 2(t_P - 14)$
1-2-4-6	3	18	11
1-2-5-6	5	15	7
1-2-4-5-6	14	14	14
1-3-2-4-6	13	13	11
1-3-2-5-6	15	10	7
1-3-2-4-5-6	24	9	14
1-3-4-6	16	17	22
1-3-4-5-6	27	13	25
1-3-5-6	24	8	12

- **Programmation Linéaire** : Le problème peut être formulé comme un programme linéaire, mais il contiendrait un nombre exponentiel de contraintes, le rendant également impraticable.
- **Algorithme du Sous-gradient** : Une méthode itérative efficace. À chaque itération k , on calcule le chemin le plus court P_k avec les poids actuels. Le multiplicateur λ est ensuite mis à jour en fonction de la violation de la contrainte : $\lambda(k+1) = \max(0, \lambda_k + \mu_k(t_{P_k} - T))$. Si le chemin P_k viole la contrainte de temps ($t_{P_k} > T$), λ est augmenté. Sinon, il est diminué. La taille du pas μ_k doit décroître de manière appropriée pour garantir la convergence (par exemple, $\mu_k \rightarrow 0$ mais $\sum \mu_k \rightarrow \infty$). La borne inférieure $L(\lambda_k)$ n'est pas garantie d'augmenter à chaque étape, mais converge vers l'optimum.

Qualité de la Borne

La borne inférieure obtenue par la relaxation lagrangienne est aussi bonne que celle obtenue par la **relaxation linéaire** du problème, où les variables x_{ij} sont autorisées à prendre des valeurs entre $[0, 1]$ au lieu de $\{0, 1\}$.

4.4. Application au Problème du Voyageur de Commerce (TSP)

Caractérisation du TSP et Relaxation par le 1-Arbre

Le TSP consiste à trouver le circuit hamiltonien de coût minimum. Un tel circuit est défini par deux contraintes principales :

1. Le degré de chaque nœud est exactement 2.
2. Les arêtes sélectionnées forment une seule composante connexe (pas de sous-tours).

La relaxation de Held-Karp pour le TSP se concentre sur la relaxation des contraintes de degré en utilisant le concept de **1-arbre**. Un 1-arbre est un graphe composé de :

1. Un arbre couvrant minimum sur le sous-graphe des nœuds $\{2, \dots, n\}$.
2. Les deux arêtes de coût le plus faible reliant le nœud 1 au reste du graphe.

Un 1-arbre a exactement n arêtes. Le problème du 1-arbre minimum est facile à résoudre. Un circuit hamiltonien est un cas particulier de 1-arbre où chaque nœud a un degré de 2. Le problème du TSP est donc équivalent à trouver le 1-arbre minimum respectant les contraintes de degré.

Formulation Lagrangienne

Pour obtenir une borne inférieure, on relaxe les contraintes de degré $\sum_{e \in \delta(i)} x_e = 2$ pour chaque nœud i . On introduit un multiplicateur de Lagrange π_i pour chaque nœud.

La fonction lagrangienne $L(\pi)$ est : $L(\pi) = \min [\sum_e x_e \cdot w_e + \sum_i \pi_i (2 - \sum_{e \in \delta(i)} x_e)]$ sujet à la contrainte que les arêtes sélectionnées forment un 1-arbre.

Cela peut être réécrit comme : $L(\pi) = \min [\sum_{e=\{i,j\}} x_e \cdot (w_e - \pi_i - \pi_j)] + 2 \sum_i \pi_i$

Le calcul de $L(\pi)$ pour un ensemble de multiplicateurs π donnés se résume à trouver un 1-arbre minimum avec des poids d'arête modifiés $(w_e - \pi_i - \pi_j)$. Le but est de trouver les π qui maximisent $L(\pi)$.

Optimisation et Preuve d'Optimalité

L'algorithme du sous-gradient est utilisé pour trouver les meilleurs multiplicateurs. La règle de mise à jour est basée sur le degré des nœuds dans le 1-arbre T_k trouvé à l'itération k : $\pi'_i \leftarrow \pi_i + \mu_k (2 - \deg(i))$. L'intuition est d'augmenter le multiplicateur (et donc de "pénaliser" les arêtes incidentes) pour les nœuds de degré trop faible (< 2) et de le diminuer pour ceux de degré trop élevé (> 2).

Une propriété cruciale émerge si l'on travaille avec des multiplicateurs dont la somme est nulle ($\sum \pi_i = 0$). Dans ce cas, le coût d'un tour (où chaque $\deg(i) = 2$) est identique dans le graphe original et dans le graphe avec les poids modifiés.

Ceci conduit à une puissante preuve d'optimalité :

Si, au cours des itérations, le 1-arbre minimum trouvé est un circuit hamiltonien (c'est-à-dire que tous les nœuds ont un degré de 2), alors ce circuit est une solution optimale pour le TSP. En effet, la borne inférieure $L(\pi)$ est égale au coût d'une solution réalisable (le tour trouvé), prouvant qu'aucune meilleure solution n'existe.

4.5.Figures Historiques et Bibliographie

Le développement de ces méthodes repose sur les travaux de plusieurs mathématiciens et informaticiens de renom :

- **Joseph-Louis Lagrange (1736-1813)** : A introduit la méthode des multiplicateurs pour l'optimisation avec contraintes d'égalité.
- **Hugh Everett III (1930-1982)** : A développé l'utilisation des multiplicateurs de Lagrange généralisés pour la recherche opérationnelle.
- **Naum Zuselevich Shor (1937-2006)** : A développé les méthodes de sous-gradient.
- **Michael Held & Richard M. Karp** : Ont appliqué la relaxation lagrangienne au TSP en utilisant la relaxation du 1-arbre dans leur article séminal. Richard M. Karp a reçu le prix Turing en 1985.

5. Programmation Linéaire et l'Algorithme du Simplexe

5.1. Résumé Exécutif

Ce document de synthèse analyse les principes fondamentaux de la programmation linéaire (PL), une méthode d'optimisation visant à maximiser ou minimiser une fonction objectif linéaire soumise à un ensemble de contraintes également linéaires. L'analyse révèle que la solution optimale d'un problème de PL se situe toujours à l'un des sommets de la région réalisable, qui forme un polytope convexe.

L'algorithme du Simplexe est présenté comme la méthode prédominante pour résoudre ces problèmes. Plutôt que d'énumérer de manière exhaustive et impraticable tous les sommets, le Simplexe parcourt intelligemment les sommets du polytope, se déplaçant d'un sommet à un voisin améliorant la fonction objectif jusqu'à ce qu'un optimum soit atteint. La mise en œuvre de cet algorithme nécessite la conversion du problème en une forme standardisée, notamment par l'introduction de "variables d'écart" pour transformer les inéquations en équations.

Pour les cas où une solution initiale n'est pas évidente, la méthode du "Simplexe à Deux Phases" est utilisée : une première phase résout un problème auxiliaire pour trouver une solution de base réalisable, et une seconde phase optimise le problème original à partir de cette solution. Bien que l'algorithme du Simplexe soit très efficace en pratique, sa complexité dans le pire des cas est exponentielle. Le document aborde également la Programmation Linéaire en Nombres Entiers (PLNE), une variante NP-difficile résolue par des techniques comme le "Branch and Bound" avec relaxation PL. Enfin, il est souligné que les applications pratiques reposent majoritairement sur des solveurs commerciaux spécialisés tels que Gurobi ou IBM CPLEX pour leur robustesse et leur efficacité.

5.2. Introduction à la Programmation Linéaire (PL)

La programmation linéaire est une technique mathématique pour optimiser un résultat, tel que le profit ou le coût, dans un modèle dont les exigences sont représentées par des relations linéaires.

Définition Formelle

Un programme linéaire consiste en deux éléments principaux :

1. **Une fonction objectif linéaire** à maximiser (ou minimiser).
2. Un ensemble de **contraintes linéaires** qui délimitent l'espace des solutions.

En notation matricielle, un problème de maximisation standard s'écrit comme suit :

- **Maximiser** : cx
- **Sous les contraintes** :
 - $Ax \leq b$
 - $x \geq 0$

Exemple Illustratif : Le Problème du Brasseur

Un exemple concret est utilisé pour illustrer les concepts de la PL : une petite brasserie cherche à maximiser son profit en produisant de l'Ale et de la Bière. La production est limitée par des ressources rares : le maïs, le houblon et le malt.

Ressources Disponibles :

- Maïs (Corn) : 480 lbs
- Houblon (Hops) : 160 oz
- Malt : 1190 lbs

Recettes et Profits par fût :

Produit	Maïs (lbs)	Houblon (oz)	Malt (lbs)	Profit
Ale	5	4	35	13 \$
Bière	15	4	20	23 \$

Formulation du Programme Linéaire :

Soit A le nombre de fûts d'Ale et B le nombre de fûts de Bière. Le problème est de :

- **Maximiser** : $13A + 23B$ (Profit total)
- **Sous les contraintes** :
 - $5A + 15B \leq 480$ (Contrainte sur le maïs)
 - $4A + 4B \leq 160$ (Contrainte sur le houblon)
 - $35A + 20B \leq 1190$ (Contrainte sur le malt)
 - $A, B \geq 0$ (Contraintes de non-négativité)

5.3. Interprétation Géométrique et Optimalité

La résolution d'un programme linéaire peut être visualisée géométriquement.

La Région Réalisable : Un Polytope Convexe

L'ensemble de toutes les solutions qui satisfont les contraintes est appelé la **région réalisable**. Cette région est un **polytope convexe**. Un ensemble est dit convexe si, pour deux points quelconques de l'ensemble, le segment de droite les reliant est entièrement contenu dans cet ensemble.

Pour le problème du brasseur, la région réalisable est un polygone défini par l'intersection des demi-plans formés par les contraintes. Les sommets de ce polygone sont les points $(0, 0)$, $(34, 0)$, $(26, 14)$, $(12, 28)$ et $(0, 32)$.

Théorème Fondamental : L'Optimalité aux Sommets

Un théorème central en programmation linéaire stipule que **la valeur maximale (ou minimale) de la fonction objectif est toujours atteinte en au moins un des sommets (vertices) du polytope.**

La preuve repose sur le fait que tout point dans un polytope peut être exprimé comme une combinaison convexe de ses sommets. Si l'optimum se trouvait à l'intérieur du polytope et non sur un sommet, il serait une moyenne pondérée des valeurs aux sommets. Or, si tous les sommets ont une valeur inférieure à cet optimum supposé, leur moyenne pondérée ne pourrait pas atteindre cette valeur, ce qui conduit à une contradiction.

Limites de l'Énumération des Sommets

Une première approche algorithmique consisterait à énumérer tous les sommets, évaluer la fonction objectif pour chacun, et choisir le meilleur. Cependant, **le nombre de sommets peut croître de manière exponentielle avec le nombre de contraintes**, rendant cette approche impraticable pour les problèmes de grande dimension. Par exemple, un hypercube en n dimensions, défini par 2^n demi-espaces, possède 2^n sommets.

5.4.L'Algorithme du Simplexe : Principes et Mécanique

L'algorithme du Simplexe, développé par George Dantzig, est une méthode bien plus efficace pour résoudre les problèmes de PL.

Concept Général

Le Simplexe est un algorithme itératif qui fonctionne comme suit :

1. Il commence à un sommet de la région réalisable (une "solution de base réalisable").
2. Il se déplace vers un sommet adjacent qui améliore la valeur de la fonction objectif.
3. Il répète ce processus jusqu'à atteindre un sommet où aucun voisin n'offre une meilleure valeur. En raison de la convexité du polytope, ce sommet est garanti d'être une solution optimale.

Formes Standards pour la Résolution

Pour faciliter l'implémentation algorithmique, les problèmes de PL sont convertis en formes standardisées.

- **Forme Canonique** : Toutes les contraintes sont des inéquations $\leq$, et toutes les variables sont non-négatives (≥ 0). Tout programme linéaire peut être converti sous cette forme.
- **Forme Standard** : Toutes les contraintes sont des équations, et toutes les variables sont non-négatives. Cette forme est obtenue à partir de la forme canonique en introduisant des **variables d'écart (slack variables)**. Une variable d'écart mesure l'excédent entre la ressource utilisée et la ressource disponible. Si une variable d'écart est nulle, la contrainte correspondante est dite "tendue" (tight).

Exemple du Brasseur en Forme Standard : On introduit les variables d'écart s_C (maïs), s_H (houblon), s_M (malt) et la variable z pour l'objectif.

- **Maximiser : z**
- **Sous les contraintes :**
 - $13A + 23B - z = 0$
 - $5A + 15B + SC = 480$
 - $4A + 4B + SH = 160$
 - $35A + 20B + SM = 1190$
 - $A, B, SC, SH, SM \geq 0$

Le Processus de Pivot

Le passage d'un sommet à un autre se fait via une opération appelée **pivotage**.

1. **Solution de Base Réalisable (SBR) initiale :** On commence avec une solution simple. Pour le brasseur, si $A=0$ et $B=0$, on a $SC=480$, $SH=160$, $SM=1190$. C'est le sommet $(0,0)$ avec un profit $z=0$. Les variables non nulles (SC, SH, SM) forment la "base".
2. **Pivot 1 :**
 - **Variable entrante :** On choisit une variable hors base (ici A ou B) qui, si augmentée, augmenterait z . On choisit B car son coefficient dans l'objectif (23) est le plus élevé.
 - **Variable sortante :** On détermine quelle variable de base atteindra zéro en premier lorsque B augmente. C'est la règle du **ratio minimum** : $\min(480/15=32, 160/4=40, 1190/20=59.5)$. Le minimum est 32, correspondant à SC . SC quitte donc la base.
 - Le pivotage consiste à réécrire le système d'équations pour que B devienne une variable de base. La nouvelle SBR est $A=0$, $B=32$, correspondant au sommet $(0, 32)$ avec $z=736$.
3. **Pivot 2 :**
 - La nouvelle fonction objectif est $z = 736 - (16/3)A + (23/15)SC$. On ne peut pas améliorer z en augmentant A (coefficient négatif). *Note : Il semble y avoir une erreur dans les calculs de la source ici, car A devrait avoir un coefficient positif pour continuer. En suivant le principe, on choisit A pour entrer dans la base. Après un nouveau pivot (avec A entrant et SH sortant), on atteint une nouvelle SBR.*
4. **Optimalité :** Le processus s'arrête lorsque tous les coefficients des variables hors base dans l'équation de l'objectif sont négatifs ou nuls. Dans l'exemple final, on atteint la solution $z = 800 - SC - 2SH$. Puisque SC et SH doivent être ≥ 0 , z ne peut pas dépasser 800. La solution optimale est donc atteinte avec $z=800$ au point $A=12, B=28$.

5.5.Défis Pratiques et Solutions Avancées

Trouver une SBR initiale : Le Simplexe à Deux Phases

L'exemple du brasseur était "chanceux" car les seconds membres (b) des contraintes étaient tous positifs, ce qui fournissait une SBR initiale évidente (toutes les variables d'écart). Si ce n'est pas le cas, on utilise le **Simplexe à Deux Phases**.

- **Phase 1 :** On cherche une SBR en résolvant un problème auxiliaire. On introduit des "variables artificielles" pour chaque contrainte problématique et on minimise leur somme. Si le minimum atteint est zéro, les variables artificielles peuvent être éliminées

et on a trouvé une SBR pour le problème original. Sinon, le problème original est infaisable.

- **Phase 2** : On applique l'algorithme du Simplexe standard au problème original, en partant de la SBR trouvée en Phase 1.

Considérations de Calcul

- **Dégénérescence et Cyclage** : Il est possible que le Simplexe stagne sur un sommet (dégénérescence) ou pire, qu'il cycle à travers plusieurs bases correspondant au même sommet sans progresser. L'utilisation de la **Règle de Bland** (choisir la variable éligible avec le plus petit indice) garantit la terminaison de l'algorithme et évite le cyclage.
- **Complexité Temporelle** : En pratique, le Simplexe est très efficace et possède une complexité moyenne polynomiale. Cependant, des exemples (Klee-Minty, 1972) montrent que sa complexité dans le pire des cas est exponentielle. Des algorithmes à complexité polynomiale prouvée existent (méthode de l'ellipsoïde, points intérieurs), mais ils ne sont pas toujours plus performants que le Simplexe en pratique.

5.6. Extensions et Applications

Programmation Linéaire en Nombres Entiers (PLNE)

La PLNE (ou ILP) est une variante où les variables sont contraintes à être des nombres entiers. Ce problème est **NP-difficile**. La méthode de résolution courante est le **Branch and Bound** :

1. On résout la **relaxation PL** (on ignore la contrainte d'intégrité). La valeur obtenue est une borne supérieure sur la solution entière.
2. Si la solution de la relaxation est entière, on a trouvé une solution candidate.
3. Sinon, on choisit une variable x_i avec une valeur non-entière v et on crée deux sous-problèmes (branches) en ajoutant les contraintes $x_i \leq \text{floor}(v)$ et $x_i \geq \text{ceil}(v)$.
4. On explore l'arbre de recherche ainsi créé, en élaguant les branches dont la borne supérieure est inférieure à la meilleure solution entière trouvée jusqu'à présent.

Applications

Tout problème qui peut être réduit à un PL peut être résolu en temps polynomial. Un exemple classique est le **problème de flot à coût minimum** dans les réseaux, qui consiste à acheminer un certain volume de flux d'une source à une destination au moindre coût, tout en respectant les capacités des arcs.

5.7. Outils et Implémentation

En pratique, il est rare d'implémenter soi-même l'algorithme du Simplexe. Le rendre efficace et numériquement stable est un art.

Solveurs Commerciaux et Académiques

Les praticiens utilisent des solveurs hautement optimisés, qui sont souvent des produits commerciaux mais gratuits pour un usage académique. Les plus connus sont :

- **Gurobi Optimization**
- **IBM CPLEX**
- **FICO Xpress**

Exemple de Modélisation (Gurobi en Python)

Le code suivant montre comment modéliser un problème de flot en utilisant l'API Python de Gurobi :

```
# Create optimization model
m = Model('netflow')

# Create variables
flow = m.addVars(commodities, arcs, obj=cost, name="flow")

# Arc capacity constraints
m.addConstrs(
    (flow.sum('*', i, j) <= capacity[i, j] for i, j in arcs), "cap")

# Flow conservation constraints
m.addConstrs(
    (flow.sum(h, '*', j) + inflow[h, j] == flow.sum(h, j, '*')
     for h in commodities for j in nodes), "node")

# Compute optimal solution
m.optimize()
```

6.Flots dans les Réseaux

6.1.Résumé Exécutif

Ce document de synthèse présente les concepts fondamentaux des problèmes de flots dans les réseaux, en se concentrant sur les problèmes de flot maximum (Max-Flow) et de coupe minimale (Min-Cut). Il explore la relation profonde qui les unit, formalisée par le théorème Max-Flow Min-Cut, qui stipule que la valeur du flot maximum d'une source à une destination dans un réseau est égale à la capacité minimale de toutes les coupes séparant cette source et cette destination.

L'algorithme de Ford-Fulkerson est présenté comme la méthode fondamentale pour résoudre ces problèmes. Son principe consiste à augmenter itérativement le flot le long de "chemins d'augmentation" dans un graphe résiduel jusqu'à ce qu'aucune amélioration ne soit plus possible. Bien que conceptuellement central, l'algorithme de base peut présenter une complexité pseudo-polynomiale dans certains cas pathologiques.

Pour pallier cette faiblesse, des améliorations garantissant une complexité polynomiale sont examinées, notamment la stratégie de mise à l'échelle des capacités (capacity scaling) et l'algorithme d'Edmonds-Karp, qui privilégie les chemins d'augmentation les plus courts. Les applications pratiques de ces concepts sont illustrées par des exemples historiques, comme l'analyse de réseaux ferroviaires durant la Guerre Froide, et des problèmes d'ordonnancement.

6.2.Problèmes Fondamentaux : Flot Maximum et Coupe Minimale

Les problèmes de flot et de coupe sont définis sur un graphe orienté et pondéré, avec un sommet source s et un sommet puits t .

Le Problème du Flot Maximum (Max-Flow)

L'objectif est de trouver le débit maximal pouvant être acheminé de la source s au puits t . Un flot est une assignation de valeurs aux arêtes qui respecte deux contraintes majeures :

- **Contrainte de Capacité** : Le flot sur chaque arête ne peut excéder sa capacité. Formellement, pour toute arête e , $0 \leq \text{flot}(e) \leq \text{capacité}(e)$.
- **Équilibre Local** : Pour chaque sommet (à l'exception de s et t), la somme des flots entrants est égale à la somme des flots sortants.

La **valeur du flot** est définie comme la somme totale des flots entrant dans le puits t . Le problème du flot maximum consiste à trouver un flot dont la valeur est maximisée.

Exemple d'un flot dans un réseau. Les labels "flot/capacité" indiquent le flot actuel et la capacité maximale de chaque arête. La valeur totale du flot arrivant au puits t est de 28.

Le Problème de la Coupe Minimale (Min-Cut)

Une **coupe s-t** est une partition des sommets du graphe en deux ensembles disjoints, A et B , tels que la source s se trouve dans A et le puits t dans B .

La **capacité d'une coupe** est la somme des capacités de toutes les arêtes qui partent d'un sommet dans A et arrivent à un sommet dans B . Les arêtes allant de B à A ne sont pas comptabilisées.

Le problème de la coupe minimale consiste à trouver une coupe s-t dont la capacité est la plus faible possible.

Exemple d'une coupe (A, B) où A est l'ensemble des sommets grisés. Sa capacité est de $10 + 8 + 10 = 28$.

6.3.Applications Concrètes

Analyse de Réseaux Stratégiques (RAND, années 1950)

Une application historique et marquante du problème de la coupe minimale a été menée par la RAND Corporation durant la Guerre Froide. Des chercheurs ont modélisé le réseau ferroviaire reliant l'Union Soviétique aux pays d'Europe de l'Est pour identifier les points de vulnérabilité. L'objectif était de déterminer la "coupe minimale" du réseau, c'est-à-dire l'ensemble minimal de liaisons ferroviaires à couper pour interrompre le plus efficacement possible l'approvisionnement. Cette analyse a permis d'identifier le "goulot d'étranglement" (bottleneck) stratégique du réseau. Le schéma de ce réseau a été déclassifié par le Pentagone en 1999.

Le réseau ferroviaire et la coupe minimale identifiée, représentant le goulot d'étranglement des approvisionnements.

Problèmes d'Ordonnancement

Le problème du flot maximum peut modéliser des problèmes d'assignation et de planification. Considérons le scénario suivant : 4 intervenants doivent donner un séminaire dans une seule salle, avec 4 créneaux horaires disponibles. Chaque intervenant fournit ses disponibilités.

Personne	Disponibilités
Alice	11h00, 13h00
Benoît	9h00, 10h00, 13h00
Clotilde	9h00, 11h00, 13h00
Dany	11h00, 13h00

Ce problème peut être résolu en le modélisant comme un réseau de flot :

- Une **source s** et un **puits t**.
- Des **nœuds** pour chaque intervenant et chaque créneau horaire.
- Une arête de s vers chaque intervenant, avec une **capacité de 1**.
- Une arête de chaque créneau horaire vers t , avec une **capacité de 1**.

- Une arête de l'intervenant x au créneau y si x est disponible à l'heure y , avec une **capacité de 1**.

Un flot de valeur 4 dans ce réseau correspond à un planning complet où chaque intervenant est assigné à un créneau unique qui lui convient. Si le flot maximum est inférieur à 4, alors un planning complet est impossible.

6.4. Le Théorème Flot-Maximum/Coupe-Minimale

Ce théorème fondamental établit un lien puissant entre les deux problèmes. Il est une manifestation du principe de dualité en programmation linéaire, où le problème du flot maximum est le primal et celui de la coupe minimale est son dual.

Théorème (Max-Flow Min-Cut) : Pour tout réseau, les trois conditions suivantes sont équivalentes pour un flot f donné :

1. Il existe une coupe dont la capacité est égale à la valeur du flot f .
2. Le flot f est un flot maximum.
3. Il n'existe aucun chemin d'augmentation par rapport à f .

Ce théorème implique que **la valeur du flot maximum est égale à la capacité de la coupe minimale**. Cette relation est prouvée en trois étapes, montrant les implications $i \Rightarrow ii$, $ii \Rightarrow iii$ et $iii \Rightarrow i$. Un élément clé de la preuve est le **lemme de la valeur du flot**, qui énonce que pour n'importe quel flot et n'importe quelle coupe, le flux net à travers la coupe est égal à la valeur du flot.

6.5. L'Algorithme de Ford-Fulkerson

L'algorithme de Ford-Fulkerson est une méthode itérative pour trouver le flot maximum dans un réseau.

Principe de Fonctionnement

1. **Initialisation** : Commencer avec un flot nul partout.
2. **Itération** : Tant qu'il existe un "chemin d'augmentation" de s à t : a. Trouver un tel chemin. b. Calculer sa **capacité goulot** (la plus petite capacité résiduelle sur le chemin). c. Augmenter le flot le long de ce chemin de la valeur de la capacité goulot.
3. **Terminaison** : L'algorithme se termine lorsqu'il n'y a plus de chemin d'augmentation possible. Le flot est alors maximum.

Le Graphe Résiduel et les Chemins d'Augmentation

Un **chemin d'augmentation** est un chemin non orienté de s à t sur lequel on peut "pousser" plus de flot. Il peut contenir :

- Des **arêtes directes** (forward edges) qui ne sont pas encore saturées (flot < capacité).
- Des **arêtes inverses** (backward edges) dont le flot n'est pas nul (flot > 0). L'utilisation d'une arête inverse revient à "annuler" une partie du flot existant pour le rediriger.

Pour trouver ces chemins, on utilise le **graphe résiduel** G_f . Pour un flot f donné, G_f contient les mêmes sommets que le graphe original, mais ses arêtes représentent les possibilités d'augmentation du flot :

- Pour chaque arête originale (u, v) , si $\text{flot}(u, v) < \text{capacité}(u, v)$, il existe une arête directe (u, v) dans G_f avec une capacité résiduelle de $\text{capacité}(u, v) - \text{flot}(u, v)$.
- Pour chaque arête originale (u, v) , si $\text{flot}(u, v) > 0$, il existe une arête inverse (v, u) dans G_f avec une capacité résiduelle de $\text{flot}(u, v)$.

Un chemin d'augmentation est simplement un chemin de s à t dans le graphe résiduel G_f .

6.6.Implémentation et Analyse de Complexité

Implémentation en Java

L'implémentation repose sur deux classes principales :

- `FlowEdge` : Représente une arête avec sa capacité et son flot. Elle inclut des méthodes pour calculer la capacité résiduelle dans les deux sens (direct et inverse) et pour mettre à jour le flot.
- `FlowNetwork` : Représente le réseau, généralement via des listes d'adjacence. Pour chaque sommet, la liste contient toutes les arêtes incidentes (directes et inverses) pour faciliter la recherche dans le graphe résiduel.

L'algorithme principal de Ford-Fulkerson implémente une boucle qui recherche un chemin d'augmentation (par exemple, via un parcours en profondeur - DFS) et, s'il en trouve un, calcule le goulot et met à jour le flot.

Complexité Temporelle et Cas Pathologiques

Si les capacités sont des entiers, chaque augmentation accroît la valeur totale du flot d'au moins 1. Le nombre d'augmentations est donc borné par la valeur du flot maximum (f^*). La recherche d'un chemin prend un temps $O(|E|)$. La complexité est donc $O(f^* \cdot |E|)$.

Cependant, cette complexité est **pseudo-polynomiale**, car elle dépend de la valeur du flot, qui peut être très grande par rapport à la taille du graphe ($|V|$ et $|E|$). Un cas pathologique bien connu montre que l'algorithme peut effectuer 200 itérations pour un flot de 200, alors que 2 itérations suffiraient avec un meilleur choix de chemins.

6.7.Améliorations de Ford-Fulkerson

Pour garantir une complexité polynomiale, plusieurs stratégies de sélection des chemins d'augmentation ont été développées.

Edmonds-Karp (Chemins les Plus Courts)

Cette variante, proposée par Jack Edmonds et Richard Karp en 1972, consiste à toujours choisir le chemin d'augmentation le plus court en termes de nombre d'arêtes. Ce choix est implémenté en utilisant un parcours en largeur (BFS) au lieu d'un DFS pour trouver les chemins dans le graphe résiduel.

- **Complexité** : $O(|V| \cdot |E|^2)$. Cette complexité est polynomiale et ne dépend pas de la valeur des capacités.

Mise à l'Échelle des Capacités (Capacity Scaling)

L'idée est de privilégier les chemins ayant une grande capacité goulot.

1. On commence avec un seuil Δ égal à la plus grande puissance de 2 inférieure ou égale à la capacité maximale U .
2. On ne considère que les chemins d'augmentation dans le graphe résiduel dont la capacité goulot est d'au moins Δ .
3. Lorsque plus aucun chemin de ce type n'existe, on divise Δ par 2 et on recommence.
4. On continue jusqu'à ce que Δ soit inférieur à 1.

- **Complexité** : $O(|E|^2 \log U)$. Cette complexité est également polynomiale.

Résumé des Complexités

Variante de Ford-Fulkerson	Stratégie de Chemin	Complexité
Standard (DFS)	N'importe quel chemin d'augmentation	$\mathcal{O}(U \cdot E ^2)$
Edmonds-Karp (BFS)	Chemin le plus court (nombre d'arêtes)	$\mathcal{O}(V \cdot E ^2)$
Mise à l'échelle (Delta Scaling)	Chemins avec capacité goulot $\geq \Delta$	$\mathcal{O}(E ^2 \log U)$
Chemin le plus "gras" (Dijkstra)	Chemin avec la plus grande capacité goulot	$\mathcal{O}(E ^2 \log U)$

U représente la capacité maximale d'une arête.

6.8.Introduction au Flot à Coût Minimum

Le problème du flot à coût minimum est une généralisation où chaque arête possède non seulement une capacité mais aussi un coût par unité de flot. L'objectif est d'envoyer une quantité de flot B de s à t en respectant les capacités, tout en minimisant le coût total.

Une méthode de résolution est **l'algorithme des chemins successifs les plus courts** :

1. Partir d'un flot nul.
2. Dans le graphe résiduel pondéré par les coûts, trouver le chemin de s à t de coût minimum (avec Bellman-Ford, car des coûts négatifs peuvent apparaître).
3. Augmenter le flot le long de ce chemin autant que possible.
4. Répéter jusqu'à ce que la quantité de flot B soit atteinte.

7.Algorithmes de Recherche Locale pour l'Optimisation

7.1.Résumé Exécutif

Ce document présente une analyse approfondie des algorithmes de recherche locale (Local Search - LS), une classe de métaheuristiques fondamentales pour la résolution de problèmes d'optimisation complexes. La recherche locale opère en améliorant itérativement une solution complète en explorant son "voisinage", c'est-à-dire un ensemble de solutions accessibles par des modifications locales.

Le principal défi de la recherche locale est d'éviter les minima locaux, des solutions qui sont meilleures que toutes leurs voisines mais qui ne sont pas des optima globaux. Pour surmonter cet obstacle, deux stratégies principales sont employées : l'élargissement du voisinage pour permettre des "sauts" plus importants dans l'espace de recherche (par exemple, les mouvements k-Opt, la recherche à large voisinage ou LNS), et l'autorisation de mouvements qui dégradent temporairement la solution pour s'échapper des bassins d'attraction locaux (par exemple, la recherche Tabou).

Le processus commence souvent par la génération d'une solution initiale via des méthodes constructives telles que les algorithmes gloutons (Greedy) ou des approches plus sophistiquées comme la recherche en faisceau (Beam Search). Le document détaille ces techniques et explore leurs applications, notamment sur le problème emblématique du voyageur de commerce (TSP).

Des études de cas sur le Sudoku et l'ordonnancement cumulatif illustrent la modélisation pratique et la mise en œuvre de la recherche locale, en soulignant l'efficacité de la recherche Tabou et l'utilité des bibliothèques de recherche locale basée sur les contraintes (CBLS) qui facilitent le développement en automatisant le calcul des violations et des deltas. Enfin, le document conclut en mentionnant d'autres métaheuristiques et les figures clés du domaine.

7.2.Méthodes Constructives : Génération de la Solution Initiale

Avant d'améliorer une solution, il faut d'abord en construire une. Les méthodes constructives assemblent une solution élément par élément, souvent à l'aide d'heuristiques.

Algorithmes Gloutons (Greedy)

Les algorithmes gloutons construisent une solution pas à pas en choisissant à chaque étape l'élément qui semble le plus approprié, sans jamais remettre en question un choix précédent.

Structure Générale de l'Algorithme Glouton :

1. **Initialisation** : Partir d'une solution partielle triviale (souvent vide).
2. **Itération** : Tant que la solution n'est pas complète :
 - Calculer le coût incrémental pour chaque élément pouvant être ajouté.

- Choisir l'élément qui optimise ce coût.
- Ajouter cet élément à la solution partielle.
- Mettre à jour l'ensemble des éléments restants.

Applications au Problème du Voyageur de Commerce (TSP) :

- **Greedy sur le Poids des Arêtes** : Choisir l'arête la moins chère qui ne crée ni sous-cycle ni sommet de degré 3.
- **Voisin le plus Proche (Nearest Neighbor)** : Partir d'une ville et ajouter systématiquement la ville non visitée la plus proche de la dernière ville ajoutée.
- **Insertion la Moins Chère (Cheapest Insertion)** : Partir d'une tournée de deux villes et insérer la ville qui nécessite le détournement minimal.
- **Insertion la Plus Éloignée (Farthest Insertion)** : Similaire à la précédente, mais en choisissant la ville avec le coût d'insertion le plus élevé, ce qui tend à créer rapidement le squelette de la tournée.

Approches Avancées

Pour pallier la "myopie" des algorithmes gloutons, des méthodes plus complexes ont été développées.

- **Recherche en Faisceau (Beam Search)** : Cette méthode est une sorte de recherche en largeur (BFS) qui contrôle la croissance exponentielle. À chaque niveau de l'arbre de recherche, elle ne conserve que les p (largeur du faisceau) meilleurs candidats. Elle explore jusqu'à une profondeur k avant de sélectionner le meilleur élément.
- **Méthode Pilote (Pilot Method)** : Cette technique évalue l'ajout potentiel d'un élément à une solution partielle en lançant une heuristique de complétion rapide (le "pilote") pour chaque possibilité. L'élément qui mène à la meilleure solution *complète* est alors définitivement ajouté.
 - **Algorithme** :
 1. Pour chaque élément e pouvant être ajouté à la solution partielle s_p .
 2. Appliquer l'heuristique pilote h sur $s_p + e$ pour obtenir une solution complète s .
 3. Conserver l'élément e qui a conduit à la meilleure solution s .
 4. Ajouter ce meilleur e à s_p et continuer.

7.3. Amélioration de la Solution : Exploration du Voisinage

La recherche locale est un processus itératif qui part d'une solution complète (souvent obtenue par une méthode constructive) et tente de l'améliorer en effectuant de petites modifications locales.

Concept de Voisinage

Le cœur de la recherche locale est la notion de **voisinage**, noté $N(s)$, qui est l'ensemble de toutes les solutions atteignables à partir de la solution actuelle s par une modification élémentaire, appelée "mouvement". L'algorithme se déplace de solution en solution au sein de cet espace de recherche, cherchant à chaque étape une amélioration.

Algorithme de Recherche Locale de base :

```
s = generate_initial_solution()
repeat
    s' = find_improving_solution_in_N(s)
    if s' exists then
        s = s'
until no improvement is found
```

Stratégies de Sélection du Mouvement

Deux stratégies principales existent pour choisir le prochain mouvement dans le voisinage :

- **Premier Mouvement Améliorant (First Improving Move) :** Le voisinage est exploré et le premier voisin s' trouvé tel que $f(s') < f(s)$ est accepté. L'exploration s'arrête et une nouvelle itération commence depuis s' .
- **Meilleur Mouvement Améliorant (Best Improving Move) :** L'ensemble du voisinage $N(s)$ est exploré pour trouver le voisin s' qui maximise l'amélioration (c'est-à-dire qui minimise $f(s')$).

Une analyse empirique pour le TSP avec un voisinage 2-Opt montre que la stratégie "First" est généralement plus rapide en termes de complexité, avec une complexité observée de $\Theta(n^2)^{.33}$ contre $\Theta(n^3)^{.25}$ pour la stratégie "Best".

Exemple : le Voisinage 2-Opt pour le TSP

Un mouvement 2-Opt consiste à supprimer deux arêtes non adjacentes d'une tournée et à reconnecter les deux chemins résultants de la seule autre manière possible, ce qui équivaut à **inverser une sous-séquence** de la tournée.

- **Intuition :** Pour un TSP euclidien, une solution optimale ne peut pas avoir d'arêtes qui se croisent. Un mouvement 2-Opt peut toujours éliminer un croisement, améliorant ainsi la solution.
- **Taille du voisinage :** $O(n^2)$

7.4.Conception et Propriétés d'un Voisinage Efficace

Le choix du voisinage est crucial pour l'efficacité d'un algorithme de recherche locale. Un bon voisinage doit idéalement posséder plusieurs propriétés, souvent contradictoires :

Propriété	Description	Application au 2-Opt pour le TSP
Connectivité	Toute solution peut atteindre une solution optimale globale via une séquence de mouvements.	Oui, le voisinage 2-Opt est connecté.
Faible Diamètre	Peu de mouvements sont nécessaires pour relier deux solutions quelconques.	Le diamètre est en $O(n)$.
Faible Rugosité	L'espace de recherche a peu de minima locaux et une structure lisse.	Difficile pour les TSP asymétriques car l'inversion de sous-tournée change les coûts.

Petite Taille	Le nombre de voisins à évaluer est faible.	La taille est en $O(n^2)$.
Évaluation Rapide	Le coût d'un voisin peut être calculé rapidement (souvent en $O(1)$ pour le delta).	Le calcul du delta est en $O(1)$, mais l'application du mouvement est en $O(n)$.

7.5. Le Défi des Minima Locaux

Un algorithme de recherche locale de base s'arrête lorsqu'il atteint un minimum local. Pour trouver de meilleures solutions, il faut des stratégies pour s'en échapper.

Solution 1 : Agrandir le Voisinage

Une façon d'échapper aux minima locaux est d'utiliser des mouvements plus puissants, capables de "sauter" par-dessus les barrières de l'espace de recherche.

- **k-Opt** : Généralisation du 2-Opt qui consiste à remplacer k arêtes. Le 3-Opt est parfois utilisé, mais sa complexité en $O(n^3)$ le rend coûteux. Le 4-Opt et au-delà sont rarement rentables.
- **Lin-Kernighan (LKH) / Chaînes d'Éjection** : Une heuristique très puissante qui implémente une forme de k-Opt séquentiel et variable. Elle construit une séquence d'ajouts et de retrais d'arêtes ("chaîne d'éjection") et accepte la modification si une amélioration est trouvée à n'importe quelle étape de la construction. C'est l'une des méthodes les plus efficaces pour le TSP.
- **Recherche à Large Voisinage (Large Neighborhood Search - LNS)** : Cette approche pousse l'idée à l'extrême. Elle consiste à :
 1. **Détruire** une partie de la solution (par exemple, en retirant un certain nombre de clients d'une tournée).
 2. **Réparer** la solution en réinsérant les éléments retirés, souvent à l'aide d'une heuristique gloutonne ou d'une méthode exacte sur le sous-problème. Cette méthode explore un très grand voisinage de manière efficace.

Solution 2 : Accepter des Mouvements non Améliorants (Recherche Tabou)

Plutôt que d'élargir le voisinage, cette approche permet de s'en échapper en acceptant des mouvements qui dégradent la solution. Pour éviter de revenir immédiatement en arrière et de cycler, elle utilise une mémoire à court terme.

- **Principe** : La recherche Tabou explore le voisinage en choisissant le meilleur voisin, même s'il est moins bon que la solution actuelle. Pour éviter le cyclage (par exemple, $A \rightarrow B \rightarrow A$), le mouvement inverse ($B \rightarrow A$) est déclaré "tabou" pour un certain nombre d'itérations (la "durée tabou").
- **Algorithme** :
 1. À chaque itération, trouver le meilleur mouvement m non tabou dans le voisinage.
 2. Appliquer le mouvement : $s = s \leftrightarrow m$.
 3. Déclarer le mouvement inverse m^{-1} tabou pour d itérations.
 4. Mettre à jour la meilleure solution trouvée jusqu'à présent si s est meilleure.

- **Paramétrage** : La durée tabou est un paramètre critique. Si elle est trop courte, le cyclage n'est pas évité. Si elle est trop longue, la recherche est trop contrainte. Une règle empirique est de choisir une durée proche de $\sqrt{|N(s)|}$. Des stratégies dynamiques ou aléatoires pour la durée tabou sont souvent efficaces.
- **Mécanismes additionnels** :
 - **Critère d'aspiration** : Permet d'accepter un mouvement tabou s'il mène à une solution meilleure que la meilleure solution globale trouvée jusqu'ici.
 - **Redémarrage (Restart)** : Si la recherche stagne, effectuer des mouvements aléatoires pour explorer une nouvelle région de l'espace de recherche.

7.6.Études de Cas

Sudoku

La recherche locale peut résoudre le Sudoku. Une modélisation efficace consiste à :

1. **Initialisation** : Remplir la grille de manière à respecter une contrainte "dure" : chaque chiffre de 1 à 9 apparaît exactement 9 fois. Les cases initiales sont fixes.
2. **Contraintes "molles"** : L'objectif est de minimiser le nombre de violations des contraintes de ligne, colonne et bloc 3x3.
3. **Voisinage** : Un mouvement consiste à échanger les valeurs de deux cellules non fixes. Pour réduire la taille du voisinage, on peut se limiter à échanger des cellules sur la même ligne (ce qui préserve les contraintes "dure" de ligne).
4. **Résolution** :
 - Une recherche **gloutonne** (best improvement) se bloque rapidement dans un minimum local.
 - Une **recherche Tabou** permet d'échapper à ces minima en acceptant des mouvements dégradants et trouve une solution avec zéro violation.

L'implémentation est grandement facilitée par les bibliothèques de **Recherche Locale Basée sur les Contraintes (CBLS)**, qui gèrent automatiquement le calcul des violations et des deltas de coût pour les mouvements.

Ordonnancement Cumulatif

Le problème consiste à ordonnancer des activités non-préemptives, chacune ayant une durée et une consommation de ressource, sur une ressource de capacité limitée C , afin de minimiser la durée totale (makespan).

L'algorithme **IFlat-IRelax** alterne deux phases :

1. **Flatten (Aplanir)** : Si le profil de consommation dépasse la capacité C , ajouter des contraintes de précédence entre les activités en conflit pour rendre la solution réalisable. Cela augmente généralement le makespan.
2. **Relax (Relâcher)** : Pour tenter de réduire le makespan, identifier le chemin critique (la séquence d'activités qui détermine le makespan) et relâcher aléatoirement certaines des contraintes de précédence sur ce chemin.

Ce cycle de réparation (flatten) et de perturbation (relax) permet d'explorer l'espace des solutions d'ordonnancement.

7.7.Conclusion et Perspectives

Ce document n'a fait qu'effleurer la surface du vaste domaine de la recherche locale. De nombreuses autres métaheuristiques existent pour améliorer ces concepts de base :

- **Recuit Simulé (Simulated Annealing)** : Accepte des mouvements dégradants avec une probabilité qui diminue au fil du temps.
- **Recherche Locale Itérée (Iterated Local Search)** : Alterne des phases de recherche locale avec des phases de perturbation.
- **Métaheuristiques basées sur une population** : Les algorithmes génétiques et mémétiques maintiennent et font évoluer un ensemble de solutions.
- **Méthodes constructives avec apprentissage** : L'optimisation par colonie de fourmis (Ant Colony Optimization) et GRASP (Greedy Random Adaptive Search) ajoutent une mémoire à long terme aux méthodes constructives.

La recherche locale est un outil puissant et polyvalent, dont l'efficacité repose sur une modélisation judicieuse du problème, une conception soignée du voisinage et le choix d'une métaheuristique adaptée pour guider l'exploration de l'espace de recherche.

8.Programmation par Contraintes (PPC)

8.1.Résumé Exécutif

La Programmation par Contraintes (PPC) est un paradigme de programmation déclaratif puissant, conçu pour résoudre des problèmes d'optimisation discrète complexes. Son principe fondamental, résumé par le slogan « **PPC = Modèle (+ Recherche)** », consiste à séparer la description du problème (le modèle) de l'algorithme utilisé pour le résoudre (la recherche). L'utilisateur définit les variables, leurs domaines de valeurs possibles et les contraintes qui régissent les solutions valides, tandis qu'un moteur de résolution, ou "solveur", explore l'espace des solutions possibles de manière efficace.

Cette approche se révèle particulièrement adaptée aux problèmes d'optimisation du monde réel, souvent qualifiés de "désordonnés" (messy), tels que la planification (scheduling), l'ordonnancement du personnel (rostering) et le routage (routing), qui comportent de nombreuses contraintes hétérogènes et des fonctions objectives non standards.

L'analyse du problème des N-Reines illustre l'évolution des techniques de résolution. Une approche naïve de "génération et test" (DFS + Filter) est rapidement écartée en raison de son inefficacité exponentielle. Une méthode plus performante, DFS + Prune, améliore considérablement la recherche en validant les contraintes sur des solutions partielles et en élaguant les branches invalides de l'arbre de recherche. Cependant, cette dernière reste spécifique au problème. La transition vers un solveur générique (Tiny-CSP) démontre la véritable puissance de la PPC : la réutilisabilité et la modularité. Bien que ce cadre générique introduise une surcharge de performance due à des mécanismes comme la gestion d'état et la propagation, il offre une base robuste pour résoudre une large gamme de problèmes. L'efficacité d'un tel solveur repose sur des composants clés, notamment un algorithme de point fixe pour la propagation des contraintes et une gestion efficace de l'état lors de la recherche. Des optimisations, comme un algorithme de point fixe piloté par les données, sont essentielles pour construire des solveurs performants capables de rivaliser avec des implémentations spécialisées.

8.2.Le Domaine de l'Optimisation Discrète

L'optimisation discrète est omniprésente dans de nombreux secteurs industriels et logistiques. Ses applications principales incluent :

- **La Planification (Scheduling)** : Allocation de ressources à des tâches sur une période donnée.
- **Le Routage (Routing)** : Détermination des itinéraires optimaux pour des véhicules, comme dans le Problème du Voyageur de Commerce (TSP).
- **L'Ordonnancement du Personnel (Rostering)** : Création d'horaires pour les employés respectant des contraintes légales, contractuelles et personnelles.

Contrairement aux problèmes académiques purs, tels que le TSP de base, les problèmes d'optimisation discrets en pratique sont qualifiés de "désordonnés" (messy). Ils se caractérisent par la présence de multiples véhicules, de dizaines de contraintes complexes et parfois de fonctions objectives inhabituelles, rendant leur résolution particulièrement difficile.

8.3.Introduction à la Programmation par Contraintes (PPC)

La PPC est une technologie de choix pour aborder ces problèmes d'optimisation complexes. Elle est présentée comme un outil polyvalent, à l'image d'un couteau suisse ou de briques LEGO, soulignant sa flexibilité et sa modularité.

Le Paradigme Déclaratif

La PPC s'inscrit dans le paradigme de la programmation déclarative, qui se concentre sur l'expression de la logique d'un calcul sans en décrire le flux de contrôle. Pour la résolution de problèmes combinatoires, cela signifie que l'utilisateur exprime les propriétés des solutions recherchées, laissant au solveur le soin de les trouver.

Cette philosophie est parfaitement résumée par la citation d'E. Freuder :

“Constraint programming represents one of the closest approaches computer science has yet made to the Holy Grail of programming: the user states the problem, the computer solves it.”

Slogan de la PPC : PPC = Modèle (+ Recherche)

Le fonctionnement de la PPC repose sur deux piliers :

- **Le Modèle** : Décrit par l'utilisateur via une API. Il contient la définition des variables du problème et des contraintes qui les lient. C'est la partie déclarative.
- **La Recherche** : La partie algorithmique, prise en charge par le solveur. Elle consiste à explorer un arbre de recherche pour trouver une ou plusieurs solutions qui satisfont toutes les contraintes du modèle.

8.4.Étude de Cas : Le Problème des N-Reines

Le problème des N-Reines, qui consiste à placer N reines sur un échiquier de $N \times N$ sans qu'aucune ne puisse en menacer une autre, sert de fil conducteur pour illustrer les concepts de la PPC.

Stratégies de Modélisation

Deux approches de modélisation sont possibles :

1. **Variables Booléennes** : Une variable booléenne par case (N^2 variables), indiquant la présence ou l'absence d'une reine. Cette approche nécessite de vérifier explicitement les contraintes sur les lignes, les colonnes et les diagonales.
2. **Variables Entières** : Une variable entière par colonne (N variables), dont la valeur $\{0, \dots, N-1\}$ indique la ligne où la reine est placée. Ce modèle est plus efficace car les contraintes de colonne sont implicitement satisfaites. Il ne reste qu'à vérifier les contraintes sur les lignes et les diagonales.

Stratégies de Résolution Évolutives

Trois approches algorithmiques sont comparées :

1. **DFS + Filter (Générer et Tester)** : Cette méthode explore toutes les assignations possibles (N^N) et ne vérifie les contraintes qu'une fois une solution complète générée. Elle est extrêmement inefficace.
2. **DFS + Prune (Élagage)** : Une amélioration majeure qui vérifie la validité des contraintes sur les solutions partielles (préfixes). Dès qu'une contrainte est violée, la branche correspondante de l'arbre de recherche est coupée ("élaguée"), évitant une exploration inutile. Bien que beaucoup plus rapide, le code produit est spécifique au problème et difficilement réutilisable.
3. **Tiny-CSP (Solveur Générique)** : L'approche PPC qui vise la généricité et la réutilisabilité. Le problème est abstrait en un ensemble de variables, de domaines et de contraintes gérés par un solveur. Le code devient plus déclaratif et les composants (moteur de recherche, types de contraintes) peuvent être réutilisés pour d'autres problèmes comme le Sudoku.

8.5. Architecture d'un Solveur PPC : Le Modèle Tiny-CSP

Un solveur PPC s'articule autour de plusieurs composants fondamentaux.

Composants Clés

Le paradigme de calcul repose sur l'interaction entre :

- **Le Magasin de Domaines (Domain Store)** : Maintient l'ensemble des valeurs possibles pour chaque variable.
- **Le Magasin de Contraintes (Constraint Store)** : Contient l'ensemble des contraintes du modèle.
- **La Recherche (Search)** : L'algorithme qui explore l'espace des solutions, généralement un parcours en profondeur (DFS).

Le Moteur de Propagation et l'Algorithme de Point Fixe

Le cœur du solveur est le moteur de propagation, qui implémente un algorithme de point fixe. Son rôle est de réduire les domaines des variables en appliquant itérativement les algorithmes d'élagage associés à chaque contrainte, jusqu'à ce qu'aucune valeur ne puisse plus être supprimée.

Un algorithme de point fixe "naïf" fonctionne comme suit :

```
fixPoint() {
  repeat
    select a constraint c;
    if c is infeasible given the domain store then
      return failure;
    else
      apply the pruning algorithm associated with c;
  until (no constraint can remove any value);
  return success;
}
```


Gestion de l'État et Recherche

Pendant la recherche, à chaque fois qu'une décision est prise (par exemple, fixer la valeur d'une variable), le solveur doit pouvoir revenir en arrière (backtrack) si cette décision mène à une impasse. Cela nécessite un mécanisme de gestion de l'état. Dans l'implémentation simple de `Tiny-CSP`, cela est réalisé en clonant l'état des domaines avant chaque branche de la recherche et en le restaurant après le backtrack.

La Recherche DFS dans `Tiny-CSP`

L'algorithme de recherche en profondeur se déroule ainsi :

1. Choisir une variable non encore fixée.
2. Si aucune n'existe, une solution est trouvée.
3. Sinon, pour une valeur v dans le domaine de la variable : a. Sauvegarder l'état actuel des domaines. b. Créer une branche en assignant la valeur v à la variable ($x = v$). c. Lancer le point fixe pour propager la contrainte. d. Appeler récursivement la recherche. e. Restaurer l'état des domaines. f. Créer la branche alternative en retirant la valeur v ($x \neq v$) et répéter les étapes c et d.

8.6. Analyse des Performances et Pistes d'Amélioration

La comparaison des trois approches pour le problème des N-Reines révèle des différences de performance significatives.

Comparaison des Approches

Approche	N	Nœuds	Temps (ms)	#Solutions
NQueensChecker	8	19 173 961	167	92
(DFS + Filter)	10	11 111 111 111	101 497	724
NQueensPrune	12	856 189	130	14 200
(DFS + Prune)	14	27 358 553	4 550	365 596
NQueensTinyCSP	12	102 531	2 439	14 200
(Solveur Générique)	14	2 934 559	72 753	365 596

Analyse :

- L'approche `NQueensChecker` est impraticable pour des tailles de N même modestes.
- `NQueensPrune` est très efficace, car son code est spécialisé et optimisé pour le problème.
- `NQueensTinyCSP` est plus lent que `NQueensPrune` en raison de la surcharge liée à son architecture générique. L'analyse de profilage montre que le temps est principalement consommé par l'algorithme de point fixe, le clonage des domaines (`backupDomains`) et leur restauration (`restoreDomains`).

Sources d'Inefficacité et Améliorations

1. **Algorithme de Point Fixe Naïf** : La principale source d'inefficacité est de ré-exécuter toutes les contraintes à chaque itération, même si leurs variables n'ont pas été modifiées.

- **Solution** : Un algorithme de point fixe amélioré, piloté par les données (data-driven). Il utilise une file d'attente pour ne propager que les contraintes dont au moins une variable a vu son domaine modifié.
- 2. **Gestion de l'État par Clonage** : La création de nombreux objets "Domain" à chaque nœud de l'arbre de recherche est coûteuse en temps et en mémoire.
 - **Solution** : Utiliser des structures de données plus efficaces pour sauvegarder et restaurer les changements de manière incrémentale, sans créer de nouveaux objets.
- 3. **Recherche Basique** : La recherche peut être améliorée avec des heuristiques plus complexes pour choisir la prochaine variable à assigner et la valeur à tester.
 - **Solution** : Implémenter un mécanisme de recherche flexible et générique.

8.7.Applications et Modélisation Avancée

La puissance expressive de la PPC la rend très efficace pour modéliser des problèmes complexes, parfois de manière plus intuitive que d'autres paradigmes comme la Programmation Linéaire en Nombres Entiers (PLNE / MIP).

Comparaison PPC vs. PLNE (Exemple du TSP)

- **Modèle PLNE** : Utilise des variables binaires x_{ij} pour indiquer si l'arc (i, j) fait partie du tour. Nécessite un nombre exponentiel de contraintes d'élimination de sous-tours.
- **Modèle PPC** : Peut utiliser un tableau de variables `succ`, où `succ[i]` est la ville qui suit la ville i . La contrainte globale `Circuit(succ)` garantit à elle seule la formation d'un tour unique. De plus, la PPC permet d'indexer des tableaux avec des variables (par exemple, pour calculer le coût total `d[i, succ[i]]`), une opération très expressive.

Projets et Perspectives

La PPC peut être appliquée à une multitude de problèmes ludiques et complexes, tels que :

- Le Carré Magique (Magic Square)
- Le Killer Sudoku
- Le Problème du Cavalier (Knight's Tour)

Le développement d'un solveur PPC plus avancé, comme **MiniCP**, permettrait d'implémenter des contraintes très utiles pour le routage et la planification, et de s'attaquer à des problèmes d'optimisation de grande envergure.

9.Algorithmes de Recherche Informée : De Dijkstra à A* Anytime

9.1.Résumé Exécutif

Ce document synthétise les principes des algorithmes de recherche de chemin informé, en commençant par l'algorithme de Dijkstra et en progressant vers des variantes avancées comme A* et Anytime Weighted A*. L'algorithme de Dijkstra, bien qu'efficace pour trouver les plus courts chemins depuis une source unique vers tous les autres sommets, explore le graphe de manière extensive et non dirigée, ce qui est sous-optimal pour trouver un chemin entre deux points spécifiques.

Pour pallier cette faiblesse, l'algorithme A* introduit une **heuristique**, une fonction qui estime le coût restant pour atteindre la destination. En combinant le coût réel depuis le départ ($g(n)$) et le coût heuristique vers l'arrivée ($h(n)$), A* priorise les chemins les plus prometteurs, réduisant considérablement le nombre de nœuds explorés. L'optimalité de A* est garantie si l'heuristique est **admissible**, c'est-à-dire qu'elle ne surestime jamais le coût réel.

Des variations comme **Weighted A*** (A* pondéré) accentuent l'influence de l'heuristique pour trouver plus rapidement des solutions, souvent au détriment de l'optimalité garantie. L'**Anytime Weighted A*** pousse ce concept plus loin, en se comportant comme un algorithme "anytime" : il trouve rapidement une première solution (potentiellement sous-optimale) puis continue d'affiner sa recherche pour trouver de meilleures solutions s'il dispose de plus de temps, utilisant les solutions trouvées pour élaguer l'espace de recherche.

Enfin, le document explore l'application de ces algorithmes à des problèmes de programmation dynamique, tels que le problème du sac à dos, en les modélisant comme des problèmes de plus court chemin sur un graphe. Cette approche soulève le défi des poids négatifs, qui invalident les hypothèses de base de Dijkstra et A*. Une version robuste de A* est nécessaire, modifiant la condition d'arrêt pour garantir l'optimalité dans de tels scénarios.

9.2.L'Algorithme de Dijkstra : Fondations et Limites

L'algorithme de Dijkstra, développé en 1959, est une méthode fondamentale pour résoudre le problème du plus court chemin depuis une source unique dans un graphe pondéré.

- **Objectif** : Partant d'un sommet source s , il calcule la distance la plus courte vers tous les autres sommets v du graphe.
- **Complexité** : En utilisant un tas binaire pour la file de priorité, sa complexité temporelle est de $O(E \log V)$, où E est le nombre d'arêtes et V le nombre de sommets.
- **Mécanisme** : L'algorithme maintient un ensemble de nœuds "ouverts" (à explorer) et un ensemble "fermé" (déjà visités). À chaque étape, il extrait de l'ensemble ouvert le nœud ayant la plus petite distance connue depuis la source et met à jour les distances de ses voisins.

Limites de Dijkstra : L'inconvénient majeur de Dijkstra est son exploration "aveugle". Il étend sa recherche uniformément dans toutes les directions sans tenir compte de la position de la

cible. Pour l'algorithme, le nœud cible est un nœud comme les autres jusqu'à ce qu'il soit atteint. Cela conduit à l'exploration d'un grand nombre de nœuds inutiles lorsque l'objectif est simplement de trouver le chemin le plus court entre un point de départ s et un point d'arrivée t .

9.3.L'Algorithme A* : L'Apport de l'Heuristique

L'algorithme A*, proposé par Hart, Nilsson et Raphael en 1968, améliore Dijkstra en utilisant une recherche informée ou "guidée". Il introduit une fonction heuristique $h(n)$ qui estime le coût du chemin le plus court entre un nœud n et la cible t .

- **Fonction d'Évaluation** : Au lieu d'explorer les nœuds en se basant uniquement sur le coût pour les atteindre depuis la source ($g(n)$), A* utilise une fonction d'évaluation $f(n) : f(n) = g(n) + h(n)$
 - $g(n)$: Le coût réel du chemin le plus court connu de la source s au nœud n .
 - $h(n)$: Une estimation heuristique du coût du chemin le plus court du nœud n à la cible t .
- **Exemple d'Heuristique** : Pour un problème de plus court chemin euclidien, l'heuristique $h(n)$ peut être la distance en ligne droite entre le nœud n et la cible.

En priorisant les nœuds avec la plus faible valeur de $f(n)$, A* explore préférentiellement les chemins qui semblent non seulement être courts jusqu'à présent, mais qui semblent aussi mener rapidement à la destination. Cela permet de focaliser la recherche et de réduire considérablement le nombre de nœuds visités par rapport à Dijkstra.

Pseudo-code de l'Algorithme A*

Paramètre	Description
Entrées	Un graphe $G=(V,E)$, une fonction de poids w , une source s , une cible t , et une heuristique $h(v)$.
Résultat	La distance la plus courte $g[t]$ de s à t .
Initialisation	$g[v]$ (coût réel) et $f[v]$ (coût total estimé) initialisés à l'infini pour tous les sommets v . $g[s]=0$, $f[s]=h(s)$.
Structures	O : Ensemble Ouvert (file de priorité min ordonnée par $f[v]$). C : Ensemble Fermé.
Boucle Principale	Tant que O n'est pas vide, extraire le nœud u avec le $f[u]$ minimum. Si u est la cible, retourner $g[t]$.
Mise à Jour	Pour chaque voisin v de u non dans C , si un chemin plus court vers v est trouvé via u , mettre à jour $g[v]$, $f[v]$ et son prédécesseur. Mettre à jour v dans O .

9.4.Propriétés des Heuristiques et Garantie d'Optimalité

La performance et la correction de l'algorithme A* dépendent crucialement des propriétés de la fonction heuristique $h(n)$.

Heuristique Admissible

- **Définition** : Une heuristique h est dite **admissible** si elle ne surestime jamais le coût réel pour atteindre la solution. C'est-à-dire, pour tout nœud n , $h(n) \leq h^*(n)$, où $h^*(n)$ est le coût réel du chemin optimal de n à la cible.
- **Théorème Fondamental** : *A* utilisant une heuristique admissible est admissible*, ce qui signifie qu'il est garanti de trouver une solution optimale (le chemin le plus court).

Une preuve par contradiction est fournie : si A* retournait un chemin sous-optimal, cela impliquerait qu'un nœud n^* sur le vrai chemin optimal n'a pas été exploré. Cependant, l'analyse des coûts $f(n^*)$ montre que ce coût serait inférieur à celui de la solution sous-optimale, ce qui est une contradiction car A* aurait dû explorer n^* en premier.

Heuristique Cohérente (ou Consistante)

- **Définition** : Une heuristique est **cohérente** si, pour chaque nœud n et chaque successeur n' , l'inégalité triangulaire est respectée : $h(n) \leq w(n, n') + h(n')$.
- **Propriété** : En pratique, il est difficile de concevoir une heuristique admissible qui ne soit pas également cohérente. L'avantage principal d'une heuristique cohérente est que lorsqu'un état est atteint pour la première fois, c'est par un chemin optimal. Il n'est donc jamais nécessaire de le ré-ajouter à la file de priorité ou de modifier sa priorité.

9.5. Variations de A* pour la Performance et les Solutions "Anytime"

A* Pondéré (Weighted A*)

Pour accélérer la recherche, surtout lorsque l'optimalité stricte n'est pas requise, on peut utiliser A* Pondéré.

- **Fonction d'Évaluation** : $f(v) = g(v) + w_h \cdot h(v)$, avec un paramètre de poids $w_h \geq 1$.
- **Comportement** : En augmentant w_h , l'algorithme devient plus "gourmand", favorisant davantage l'heuristique. Il explore moins de nœuds et trouve une solution plus rapidement. Cependant, si $w_h > 1$, la garantie d'optimalité est perdue. La solution trouvée est bornée par un facteur w_h de la solution optimale.

Anytime Weighted A*

Un algorithme "anytime" est capable de fournir des solutions de haute qualité même lorsqu'il est interrompu avant sa complétion. Anytime Weighted A* (Hansen & Zoo, 2007) est conçu pour ce comportement.

Il apporte trois modifications principales à A* pondéré :

1. **Utilisation d'une fonction d'évaluation non-admissible** ($f'(n) = g(n) + w \cdot h(n)$ avec $w \geq 1$) pour guider la recherche agressivement vers une première solution.
2. **Recherche continue après la première solution** : Une fois qu'une solution (un chemin vers t) est trouvée, son coût $g(t)$ est stocké comme une borne supérieure (solution "incumbent"). La recherche se poursuit pour trouver de meilleures solutions.

3. **Élagage par borne inférieure** : L'algorithme utilise simultanément une fonction d'évaluation admissible ($f(n) = g(n) + h(n)$) comme borne inférieure. Si $f(n)$ pour un nœud n est supérieur au coût de la meilleure solution déjà trouvée, ce chemin peut être élagué ("pruned").

9.6.Application à la Programmation Dynamique : Le Problème du Sac à Dos

Les algorithmes de type A* peuvent être utilisés pour résoudre des problèmes de programmation dynamique. Le problème du sac à dos (Knapsack Problem) en est un exemple classique.

- **Formulation** : Maximiser la valeur totale des objets sélectionnés tout en respectant une contrainte de capacité (poids total).
- **Modélisation en Plus Court Chemin** : Le problème peut être reformulé comme un problème de minimisation (minimiser la valeur négative) et résolu comme la recherche du plus court chemin dans un graphe orienté acyclique (DAG). Chaque état du graphe représente une décision sur un objet et la capacité restante.
- **Heuristique pour le Sac à Dos** : Une heuristique admissible et efficace pour ce problème est la **relaxation linéaire**, où l'on suppose qu'il est possible de prendre des fractions d'objets.

La résolution du problème du sac à dos avec A* et cette heuristique explore beaucoup moins d'états que la programmation dynamique classique.

9.7.Défis et Adaptations : La Gestion des Poids Négatifs

La transformation d'un problème de maximisation en un problème de minimisation (en utilisant des valeurs négatives) introduit des arêtes de poids négatif dans le graphe.

- **Problème** : L'algorithme de Dijkstra ne fonctionne pas avec des poids négatifs. A* standard peut également échouer car son hypothèse fondamentale (que les coûts de chemin ne font qu'augmenter) est violée.
- **Solution : Robust A*** : Pour gérer les poids négatifs dans un DAG, A* doit être modifié. Il ne peut plus s'arrêter dès qu'il atteint la cible pour la première fois. La recherche doit continuer jusqu'à ce que l'une des conditions suivantes soit remplie :
 1. L'ensemble des nœuds ouverts est vide.
 2. La valeur f minimale dans l'ensemble ouvert est supérieure ou égale au coût de la meilleure solution déjà trouvée (l'incumbent).

Cette modification garantit que même si un chemin initial vers la cible est trouvé, l'algorithme peut encore découvrir un chemin plus court via un nœud non encore exploré qui utilise une arête à poids négatif.

9.8.Aperçu du Projet de Développement

Un projet pratique est décrit, visant à appliquer ces concepts.

- **Objectifs Principaux :**

1. Implémenter l'algorithme **Anytime Weighted A***.
2. Modéliser et résoudre le **Problème du Voyageur de Commerce (TSP)** en utilisant A* et Anytime Weighted A*.
3. Développer une **heuristique h efficace pour le TSP**.

Le contexte source fournit des extraits de code Java illustrant la modélisation du problème du sac à dos, incluant la définition d'un état (`KnapsackState`), une interface de modèle générique, et l'implémentation de l'heuristique de relaxation linéaire. Ces éléments servent de base pour les implémentations requises par le projet.